

青 岛 科 技 大 学

移动软件开发课程论文

题 目 智慧商城

计算 214 徐祥禹 2101010216

信息学院 学院 计算机科学与技术 专业

2024 年 7 月 7 日

智慧商城

摘要

智慧商城小程序是一款基于微信小程序框架开发的电商平台，旨在为用户提供便捷、高效的购物体验。项目的开发背景源于电子商务的快速发展和智能手机的普及，用户对移动购物的需求日益增加。然而，现有的电商平台在用户体验、个性化推荐、虚拟试穿等方面存在不足。基于此，本项目结合了 AR 技术、AI 算法和高并发处理等前沿技术，旨在提升用户购物体验并促进实体经济的发展。

在需求分析阶段，项目详细分析了系统的功能需求、性能需求、安全需求和用户体验需求，并绘制了顶层用例图和系统流程图。系统架构设计采用分层架构，包括小程序端、视图层、业务层和数据层，各层次独立且相互协作，增强了系统的可维护性和可扩展性。功能模块设计覆盖用户、商家和管理员三大角色，提供了商品管理、订单管理、同城购物、虚拟试穿、实时客服聊天、直播等丰富的功能。

关键算法和模型部分重点介绍了协同过滤算法和基于 ResNet 的图像检索推荐算法，用于个性化商品推荐。同时，项目集成了 VisionKit 实现虚拟试鞋功能和改进的 VTON 模型实现虚拟试衣功能，通过 AR 和 AI 技术大幅提升用户的购物体验。

在界面设计和项目实现方面，智慧商城小程序注重用户界面的美观性和操作的便捷性，采用 Spring Boot 和 Flask 分别处理主要业务逻辑和虚拟试穿功能。系统测试部分进行了详细的功能测试和性能测试，确保各模块正常运行和系统高效性能。

本项目的创新点在于集成 AR 虚拟试鞋和 AI 虚拟试衣功能，为用户提供逼真的试穿体验；使用协同过滤和基于 ResNet 的图像检索推荐算法，提供个性化商品推荐；实现同城购物功能，支持快递配送和到店自取，促进实体经济发展；以及利用 WebSocket 实现用户与客服之间的双向沟通。实现直播功能，提高用户与商家互动的便利性。通过本项目的实施，不仅提升了用户的购物体验，也为商家提供了更加多样化的展示和销售方式，具有较高的商业应用价值和推广前景。

关键词：协同过滤算法；ResNet；高并发；AR；AI；直播；WebSocket

Smart Mall

Abstract: The Smart Mall Mini Program is an e-commerce platform developed based on the WeChat Mini Program framework, aiming to provide users with a convenient and efficient shopping experience. The project's development is driven by the rapid growth of e-commerce and the widespread adoption of smartphones, leading to increasing demand for mobile shopping solutions. However, existing e-commerce platforms are lacking in user experience, personalized recommendations, and virtual try-on capabilities. To address these shortcomings, this project integrates cutting-edge technologies such as AR, AI algorithms, and high-concurrency processing to enhance user shopping experiences and stimulate physical economy growth.

During the requirements analysis phase, the project thoroughly examined functional requirements, performance criteria, security needs, and user experience expectations, accompanied by top-level use case diagrams and system flowcharts. The system architecture follows a layered design encompassing the mini-program frontend, presentation layer, business logic layer, and data access layer, ensuring independence and collaboration among layers to enhance maintainability and scalability. Functional module design caters to users, merchants, and administrators, offering diverse features including product management, order processing, local shopping, virtual try-ons, real-time customer service chat, and live streaming.

Key algorithms and models focus on collaborative filtering and ResNet-based image retrieval for personalized product recommendations. Additionally, the integration of VisionKit enables virtual shoe try-ons, and the enhanced VTON model facilitates virtual clothing try-ons,

significantly elevating the shopping experience through AR and AI technologies.

In terms of interface design and implementation, the Smart Mall Mini Program emphasizes aesthetic user interfaces and intuitive interactions. Spring Boot and Flask are employed to handle core business logic and virtual try-on functionalities respectively. Rigorous functional and performance testing ensures the seamless operation and efficient performance of all modules.

The project's innovations lie in its integration of AR virtual try-ons and AI virtual clothing try-ons, providing realistic fitting experiences. It also utilizes collaborative filtering and ResNet-based image retrieval for personalized product recommendations, supports local shopping with delivery and in-store pickup options, fosters physical economy development, and enhances user engagement through real-time customer service chat and live streaming functionalities. By implementing this project, it not only enhances the shopping experience for users but also provides merchants with diversified display and sales methods, offering substantial commercial value and promising promotional prospects.

Key words : UBCF ; ResNet; High Concurrency; AR; AI; Live Streaming; Realtime Customer Service

目录

前言	1
第 1 章 绪论	1
1.1 项目背景	1
1.2 研究意义	1
1.3 发展动态	2
1.3.1 国内外发展状况	2
1.3.2 前沿技术	2
1.4 现有微信小程序电商平台的不足	2
1.5 本项目的创新点和目标	3
1.6 项目分工	4
第 2 章 需求分析	5
2.1 可行性分析	5
2.2 系统需求分析	6
2.2.1 功能需求	6
2.2.2 性能需求	8
2.2.3 安全需求	8
2.2.4 用户体验需求	9
2.3 顶层用例图	10
2.4 系统流程图	11
第 3 章 数据库设计	12
3.1 数据库表的设计	12
3.2 E-R 图	17
3.3 数据流图	17
第 4 章 系统架构与功能实现	19
4.1 系统架构设计	19
4.2 功能模块设计	20
4.3 系统主要功能实现	22

4.3.1 WebSocket 实现客服聊天	22
4.3.2 商品购买	27
4.3.3 地址管理	29
4.3.4 首页	29
4.3.5 商品列表页	31
4.3.6 筛选商品	31
4.3.7 商品详情页	32
4.3.8 收藏商品	34
4.3.9 虚拟试衣	35
4.3.10 购物车	36
4.3.11 同城购物	37
4.3.12 商品评价	38
4.3.13 订单页面	39
4.3.14 个人信息	40
第 5 章 关键算法和模型	42
5.1 VisionKit 虚拟试鞋	42
5.2 VOfitFusion(改进的-VTON 模型)	47
5.2.1 模型概述	47
5.2.2 实现方法	50
5.2.3 训练策略	50
5.2.4 实验	51
第 6 章 项目实施与使用	52
6.1 项目实施	52
6.2 系统使用手册	53
6.3 项目效果	56
第 7 章 系统测试	64
7.1 测试目的	64
7.2 测试环境	64
7.3 功能测试	64

7.4 性能测试	71
7.5 总结	71
第8章 总结	72
8.1 克服的困难	72
8.2 升级演进	73
8.3 商业推广	73
8.4 感悟	74

前言

随着电子商务的迅猛发展，智慧商城小程序应运而生，旨在通过微信小程序平台为用户提供便捷的购物体验。项目通过集成创新技术如智能推荐、AR 试鞋、AI 换衣、实时客服聊天、直播模块以及同城购物功能，不仅提升了用户的互动体验和购物便利性，还促进了实体店经济和相关服务业的发展，进一步满足了用户多样化和个性化的消费需求。

第 1 章 绪论

1.1 项目背景

随着电子商务的迅猛发展，网络购物已经成为人们日常生活中的重要组成部分。智能手机的普及和移动互联网技术的进步，使得移动端购物平台得到了广泛的应用和推广。微信小程序作为一种轻量级的应用形态，凭借其无需安装、使用便捷、覆盖广泛等优势，逐渐成为电商平台的重要载体。智慧商城小程序应运而生，旨在为用户提供更加便捷和个性化的购物体验，同时为商家提供高效的商品管理和营销工具。本项目基于微信小程序平台，利用现代信息技术和人工智能算法，实现了多种创新功能，满足用户和商家的多样化需求。

1.2 研究意义

智慧商城小程序的研究和实现具有重要的理论和实际意义。在理论上，它丰富了电子商务系统的功能和架构设计，为未来电商系统的开发提供了新的思路 and 参考。在实际应用中，它为用户提供了更便捷、更智能的购物方式，提高了用户的购物体验，增强了用户粘性。同时，为商家提供了高效的商品管理和营销工具，促进了商家销售额的增长，具有显著的经济效益。同城购物功能的实现，不仅能够刺激本地消费，增加实体店的客流量，还可以带动网约车和外卖行业的发展，创造更多的就业机会，具有重要的社会效益。

1.3 发展动态

1.3.1 国内外发展状况

国内外电子商务平台在智能化和个性化服务方面均取得了一定的进展。例如，国内的淘宝、京东等平台已经开始应用大数据和人工智能技术提升用户体验；国外的 Amazon 也在不断优化其推荐算法和用户界面设计。同时，同城购物和本地化服务也逐渐成为电商平台的发展方向。此外，直播带货也成为电商平台新的增长点，通过直播互动增加用户粘性和购物体验。

1.3.2 前沿技术

1. **人工智能技术**：如机器学习、深度学习在推荐系统中的应用。
2. **增强现实（AR）技术**：在虚拟试衣、试鞋等场景中的应用。
3. **大数据技术**：用于用户行为分析和个性化推荐。
4. **本地化服务技术**：用于支持同城购物、快递配送和到店自取等功能。
5. **直播技术**：通过实时视频互动，实现直播带货功能，提高用户购物的参与感和即时性。

1.4 现有微信小程序电商平台的不足

目前，主流电商平台在用户体验和技术应用方面存在一些不足之处：

1. **个性化推荐不足**：传统的推荐系统主要依赖于用户的历史购买记录，推荐的准确性和实时性不足。
2. **缺乏互动性**：用户在购物过程中缺乏互动体验，如无法进行虚拟试衣、试鞋等操作。
3. **用户体验不佳**：复杂的操作流程和繁琐的界面设计，影响了用户的购物体验。
4. **同城购物支持不足**：现有平台对本地商品的支持和推广力度不够，用户难以找到本地优质商品。

-
5. **商家管理不便：**商家在商品管理、订单处理和用户互动等方面的工具和支持不足。
 6. **直播功能有限：**现有平台的直播功能无法充分满足商家和用户的互动需求，直播带货效果有限。

1.5 本项目的创新点和目标

创新点：

1. **虚拟试衣、试鞋功能：**利用增强现实（AR）技术，实现虚拟试衣、试鞋功能，提升用户购物体验。
2. **商品推荐算法：**基于协同过滤算法和 ResNet 图像检索算法，为用户提供精准的商品推荐。
3. **客服聊天：**实现使用 WebSocket 和客服聊天功能，为用户提供实时、高效的客服服务体验。
4. **同城购物功能：**支持用户浏览同城商品，选择快递配送或到店自取，满足用户多样化需求，并促进实体店经济、网约车和外卖行业的发展。
5. **高效的后台管理系统：**为商家提供商品管理、订单管理、直播等功能，提高商家运营效率。
6. **直播带货功能：**商家可以通过直播实时展示和销售商品，用户可以通过直播互动即时购买，提高购物体验和商家销售额。
7. **性能优化：**为提高系统的响应速度和并发能力，后端引入了 Redis 和 RabbitMQ 来进行缓存和消息队列处理。在推荐系统中，使用 Redis 缓存推荐结果，以减少相似度的重复计算，提升系统响应速度。在支付系统中，使用 RabbitMQ 进行异步下单处理。通过将订单请求放入消息队列，异步处理订单生成和支付操作，提高系统的并发处理能力，降低高并发场景下的系统负载。

目标：

1. 提供一个功能全面、操作简便的移动端购物平台，提升用户的购物体验。

-
2. 为商家提供高效的商品管理和营销工具，促进销售额增长。
 3. 利用先进的技术手段，实现个性化推荐、虚拟试衣等创新功能，增强平台竞争力。
 4. 推动同城购物，支持本地经济发展，带动相关行业如网约车和外卖的增长。
 5. 实现直播带货功能，提升用户购物的即时性和互动性，增加用户粘性。

1.6 项目分工

在智慧商城小程序的开发过程中，项目团队由两名同学组成。为了确保项目的高效进行和任务的合理分配，根据各自的技术特长和兴趣，进行了以下任务分解：

计算 214 徐祥禹主要负责：

1. **需求分析：**明确系统的功能需求和非功能需求，绘制顶层用例图、系统流程图、时序图、功能模块图并撰写报告。
2. **UI 设计：**确保各页面和功能模块的用户体验良好，设计并优化用户界面。
3. **前端开发：**使用微信小程序框架进行小程序的页面设计与开发。实现各个功能模块的视图展示，包括商品展示页、商品详情页、购物车、订单页面、个人中心等。实现用户交互功能，如用户登录注册、商品筛选与排序、购物车操作、订单操作等。整合 AR 试鞋和 AI 换衣功能，确保前端的用户体验。
4. **算法设计与优化：**实现关键算法如虚拟试衣算法，并进行优化。使用 Flask 框架处理虚拟试穿功能，确保 AR 试鞋和 AI 换衣功能的实现。
5. **同城购物模块：**设计并实现同城购物功能，包括商品展示、快递配送和到店自取选项。
6. **实时客服聊天：**WebSocket 实现实时客服聊天功能。
7. **功能与性能测试：**设计并实施功能测试和性能测试，确保各功能模块的正常运行和系统的高效性能。

计算 213 刘洋主要负责：

1. **系统架构设计：**提出系统的总体架构设计，绘制系统架构图并撰写报告。
2. **数据库设计与实现：**使用 MyBatis 进行数据库操作，设计并实现数据库结构和

主要数据表。确保数据的增删改查功能，优化数据库性能。使用 MySQL 存储核心数据，Redis 进行数据缓存。绘制 E-R 图、数据流图。

3. **后端开发：**使用 Spring Boot 框架进行主要业务逻辑的实现，包括用户管理、商品管理、订单管理等。实现与前端的 API 接口，保证数据传输的稳定与安全。
4. **算法设计与优化：**实现关键的推荐算法。实现协同过滤算法，根据用户的订单信息计算相似用户并推荐商品。使用基于 ResNet 的图像检索推荐算法，提取用户浏览过的商品图片特征，推荐视觉上相似的商品。
5. **直播模块：**实现腾讯云服务的直播功能，包括推流地址和视频接收的实现。
6. **系统部署与配置：**负责项目的部署与配置，确保系统在不同环境中的正常运行。编写系统安装与配置指南、系统使用手册及各类用户操作说明。

第 2 章 需求分析

进行需求分析是系统开发中不可或缺的环节，它有助于明确项目目标和范围，识别用户的实际需求和期望，制定合理的开发计划，降低开发风险。通过需求分析，项目团队能够确保所有参与者对项目的期望一致，深入了解用户痛点和需求，从而开发出真正满足用户需求的产品。这一步骤还为后续开发提供了基础数据和依据，有助于识别和预防潜在风险，确保项目顺利进行并提高成功率。

2.1 可行性分析

本文从技术、经济、市场和社会四个方面进行全面可行性评估。

在技术可行性方面，微信小程序作为一种轻量级应用，依托于微信庞大的用户基础和成熟的技术生态，具备良好的技术支持。小程序开发技术门槛较低，开发工具和资源丰富，开发团队具备相应的技术能力。此外，利用先进的人工智能算法和增强现实技术，能够实现个性化推荐和虚拟试衣等功能，具备较高的技术可行性。

从经济角度来看，开发智慧商城小程序的成本主要包括开发成本、运营维护成本和市场推广成本。与传统的电商平台相比，小程序的开发和维护成本相对较

低，且用户无需下载安装，降低了推广成本。通过平台的运营和商品销售收入，能够有效覆盖开发和运营成本，实现经济上的可行性。

市场方面，随着移动互联网的普及和用户购物习惯的改变，移动端购物已成为主流趋势。微信作为国内最大的社交平台，拥有庞大的用户群体，为智慧商城小程序提供了广阔的市场基础。通过个性化推荐、虚拟试衣、同城购物等创新功能，能够满足用户多样化的需求，具有较高的市场可行性。

从社会效益来看，智慧商城小程序的推出能够提升用户购物体验，促进本地经济发展，带动相关行业如网约车和外卖行业的增长。同时，项目的实施能够为商家提供更高效率的运营工具，提升商家的经济效益和市场竞争力，具有较高的社会可行性。

2.2 系统需求分析

2.2.1 功能需求

1. 用户功能

(1) 用户注册和登录

用户可以通过微信授权登录、账号密码登录。

提供退出登录和用户注册功能。

(2) 个人中心

修改密码、姓名等个人信息。

管理收货地址，包括添加、删除和修改地址。

查看和管理订单，包括全部订单、未支付订单、待发货订单和待收货订单。

查看收藏的商品，取消收藏。

(3) 商品浏览和购买

浏览首页的商品帖子。

根据大类和小类筛选商品。

根据上架时间、价格、销量等排序商品。

通过价格区间筛选商品。

浏览商品详情，查看商品规格。

添加商品到购物车。

选择收货地址进行购买。

进行支付。

(4) 商品评价

对已收货的商品进行评分（1-5）和文字评价。

对评价内容进行文本检验，确保无不合法信息。

(5) 商品推荐

根据订单信息使用协同过滤算法推荐相似用户购买过的商品。

利用 ResNet 神经网络对用户浏览过的商品图片进行特征提取，推荐视觉上相似的商品。

(6) 虚拟试穿

提供虚拟试鞋功能。

使用 VOfitFusion 模型实现虚拟试衣，用户上传照片后展示穿上当前商品的效果。

(7) 文件管理

上传和加载文件。

(8) 优惠券

管理和使用优惠券。

(9) 同城购物

浏览同城商品，选择快递配送或到店自取。

(10) 直播功能

观看商家的直播。

2. 商家功能需求

(1) 商品管理

管理商品信息，包括上传、修改、删除商品。

上传商品图片时进行检测。

(2) 订单管理

包括查看订单详情、处理发货等。

(3) 直播管理

发起和管理直播活动。

3. 管理员功能需求

(1) 用户管理

管理用户信息。

(2) 商家管理

管理商家信息。

(3) 首页管理

管理首页的商品帖子，包括审核和删除。

2.2.2 性能需求

1. 响应时间要求：

用户在浏览商品、下单、支付等操作时，系统应保证响应时间在 1 秒以内，以提供良好的用户体验。

2. 并发用户数：

系统应支持高并发访问，尤其是在促销活动或特定时间段内的用户访问高峰期，能够稳定处理大量请求。

3. 吞吐量：

系统应能处理每秒数百甚至数千次的请求，以确保在高流量情况下的稳定性和可靠性。

4. 资源利用率：

合理利用服务器资源，通过优化数据库查询、缓存机制和异步处理等手段，降低系统负载，提高系统整体效率。

2.2.3 安全需求

1. 用户数据安全：

用户密码和个人信息应使用加密存储，确保数据在传输和存储过程中的安全性。

使用 HTTPS 协议保护用户在网站和小程序间的数据传输安全。

2. 访问控制：

用户和管理员应有明确的访问权限控制，保证只有授权人员能够访问相应的功能模块和数据。

对于敏感操作（如支付、订单管理等），应实现双重验证或其他安全措施以防止未授权访问。

3. 数据备份与恢复：

定期对重要数据进行备份，确保在系统发生故障或数据丢失时能够快速恢复服务并保持数据完整性。

4. 代码安全性：

确保后端代码和数据库查询免受 SQL 注入、XSS 攻击等安全漏洞的影响。

对上传的文件进行检测和过滤，防止恶意文件上传和执行。

2.2.4 用户体验需求

1. 界面友好性：

设计简洁直观的用户界面，使用户能够快速找到所需功能和信息，减少学习成本。保持页面布局清晰、操作简单，提供一致的用户交互体验。

2. 快速响应：

系统应保证快速的响应速度和流畅的操作体验，避免长时间的加载和等待，特别是在移动网络环境下。

3. 个性化推荐：

基于用户的历史行为和偏好，提供个性化的商品推荐，增强用户对推荐内容的认可度和购买欲望。

利用协同过滤算法分析用户的订单信息，推荐相似用户购买过的商品。

使用 ResNet 神经网络对用户浏览过的商品图片进行特征提取，推荐视觉上相似的商品。

4. 虚拟试穿功能：

提供虚拟试鞋功能，用户可以在购买鞋子前通过 AR 技术进行虚拟试穿，增强购物体验。

使用 VOfitFusion 模型实现虚拟试衣功能，用户上传照片后可以看到自己穿

上当前商品（如衣服、裤子）的效果，提升购物的趣味性和便利性。

5. 同城购物：

提供同城购物功能，用户可以浏览同城的商品，并选择快递配送或到店自取。

增加实体店经济，用户到店购物可以选择打车服务，促进网约车发展，或者选择外卖到家服务，促进骑手发展。

6. 实时反馈和客服聊天：

提供即时的操作反馈，例如订单状态变更、支付结果确认等，让用户能够及时了解操作的结果和状态。

7. 直播功能：

提供直播功能，用户可以观看商家的直播，了解商品详情和促销信息，增强互动性和购物体验。

2.3 顶层用例图

系统的顶层用例图展示了智慧商城的主要功能模块和用户角色的交互关系。该用例图包括三个主要角色：普通用户、商家和管理员。普通用户能够完成注册、登录、浏览商品、加入购物车、下单购买、商品评价、收藏商品、管理地址和支付等功能。商家则负责管理商品信息、处理订单并发起直播。管理员可以管理用户信息、商家信息以及首页帖子。通过顶层用例图，可以直观地看到各角色与系统功能之间的关系，帮助理清系统的主要需求和功能分布，为详细设计和开发提供指导。

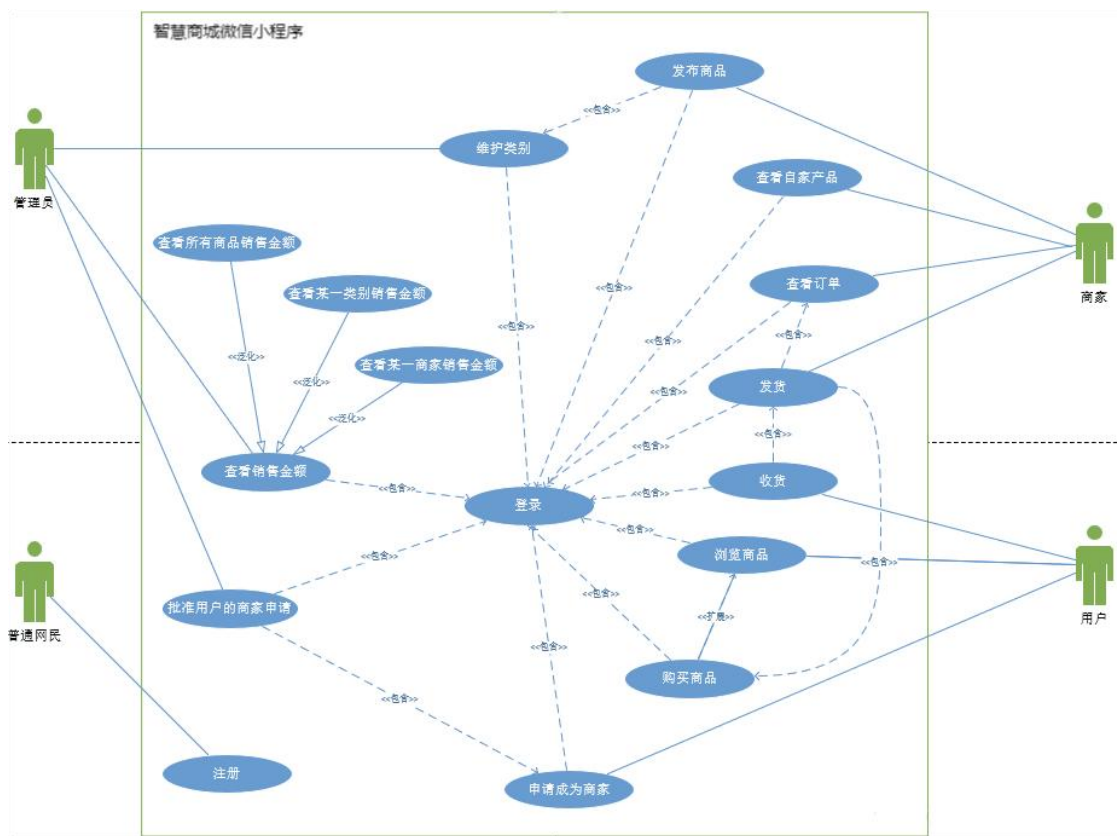


图 2-1 顶层用例图

2.4 系统流程图

系统流程图展示了智慧商城中用户和商家的主要业务流程。流程图涵盖了用户从注册、登录开始，浏览商品、选择商品规格、加入购物车、下单、选择地址、支付，再到订单生成、发货、收货和评价的全过程。同时，流程图还展示了商家管理商品信息、处理订单和发起直播的操作。每个流程节点细致地展现了数据如何在用户、前端界面、后台服务和数据库之间流转，确保系统能够顺利、高效地完成各项操作。通过系统流程图，能够清晰地理解各业务流程的逻辑顺序和数据流动，为系统开发和优化提供依据。

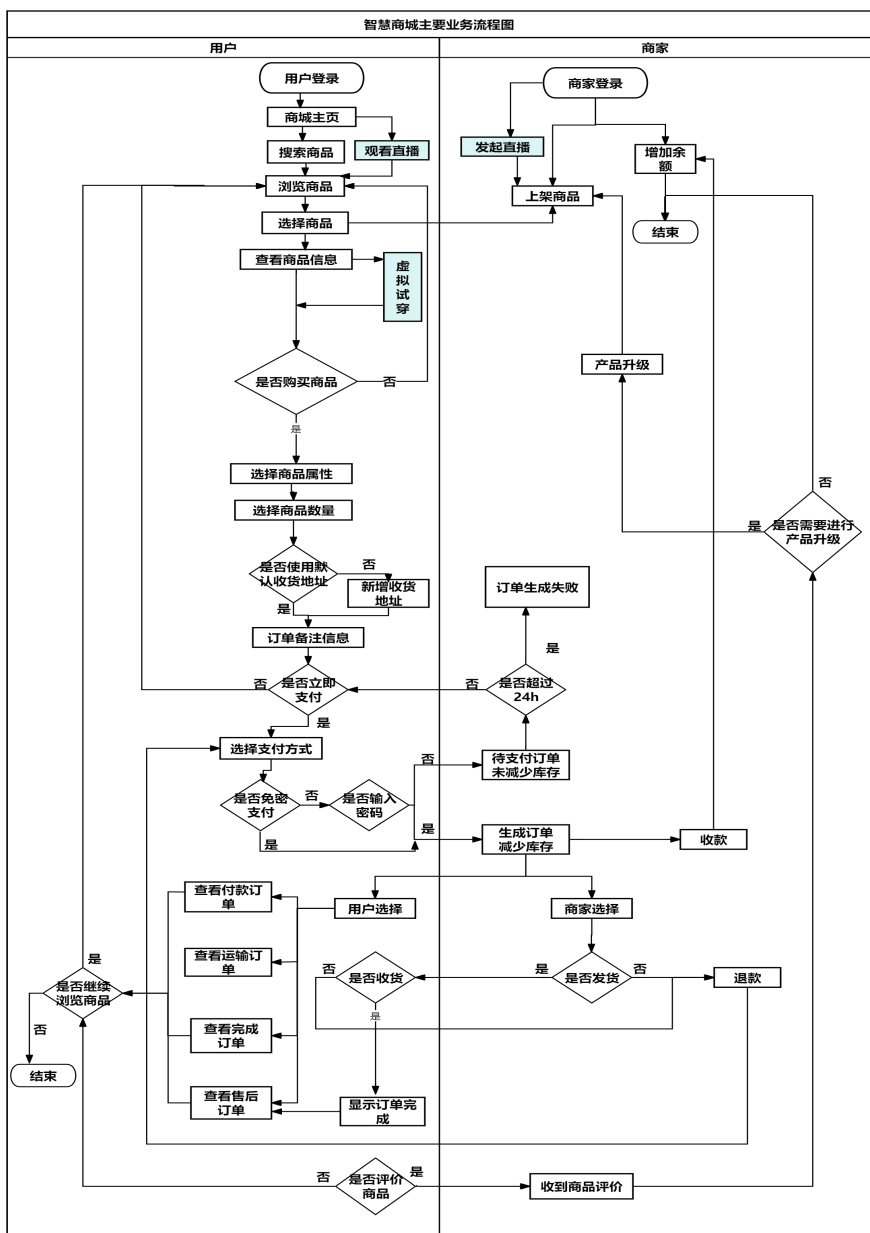


图 2-2 系统流程图

第 3 章 数据库设计

3.1 数据库表的设计

该数据库用于支持智慧商城小程序的运行，涵盖了用户、商品、订单等核心数据。数据库表协同工作，为智慧商城小程序提供了全面的数据支持，确保用户可以方便地进行商品浏览、购买、评价和收藏等操作。数据库包含以下主要表：

表 3-1address

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	id
userid	int			否	用户 id
linkname	varchar(255)			是	收货人姓名
linkaddress	varchar(255)			是	收货地址
linkphone	varchar(11)			是	收货手机号

表 3-2 cart

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	购物车 id
userid	int			否	用户 id
goodid	int			否	商品 id
standardid	int			否	规格 id

表 3-3 cate1

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	大类 id
name	varchar(255)			否	大类名称

表 3-4 cate2

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	小类 id
name	varchar(255)			否	小类名称
cate2img	varchar(255)			是	为了在分类里选择 图片+名称

cateid1	int	否
---------	-----	---

表 3-5 comment

字段	类型	主键	外键	允许空值	备注
id	int	主键			商品评价 表
userid	int				
goodid	int				
orderid	int				
star	int				评分等级 1-5 颗星
description	text				评价描述

表 3-6 favorite

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	id
goodid	int			否	收藏的商 品 id
userid	int			否	用户 id
lovetime	datetime			是	

表 3-7 good

字段	类型	主键	外键	允许空值	备注
id	bigint	主键			
cateid1	int			否	所属大类
cateid2	int			否	所属小类
name	varchar(255)			是	商品名

description	varchar(255)	是	商品描述
imgs	varchar(255)	是	图片路径
createtime	datetime	是	创建时间
salenum	int	是	总销售量
salemoney	int	是	总销售额
detail	varchar(255)	是	详情图片 商品 各角度的图片
shopid	int	是	商品所属店铺
model	varchar(255)	是	3D 模型位置

表 3-8 order

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	订单 id
orderno	varchar(255)			否	订单编号
userid	int			否	用户 id
goodid	int			否	商品 id
standardid	int			否	规格 id
createtime	datetime			是	订单创建时间
status	varchar(255)			是	订单状态 未支付 已支付 已发货 已收货
linkname	varchar(255)			是	收货人姓名
linkaddress	varchar(255)			是	收货地址
linkphone	varchar(11)			是	收货手机号
addressid	int			否	地址信息 id

表 3-9 standard

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	规格 id

goodid	int	否	商品 id
price	float	是	价格
store	int	是	库存
value	varchar(255)	否	规格描述

表 3-10 user

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	用户 id
name	varchar(255)			是	用户名字/商家店铺名称/管理员名称
phone	varchar(11)			是	手机号
password	varchar(255)			是	登录密码
description	varchar(255)			是	个性签名/店铺描述
role	varchar(50)			是	身份 消费者/商家/管理员 user/shop/admin
avatar	varchar(255)			是	头像

表 3-11 message

属性名	类型	主键	外键	空值	备注
sender	int			是	发送者 ID
receiver	int			是	接收者 ID
msg	varchar(1024)			是	消息内容
time	varchar(1024)			是	消息发送时间
flag	int			是	消息标志
id	int	是		否	消息 ID (自增)

3.2 E-R 图

智慧商城的 ER 图通过图形化展示系统中的主要实体及其相互关系，明确了如用户、商品、订单等实体及其属性，帮助开发者和设计者直观理解数据库结构。它清晰定义了各实体之间的一对多、多对多等关系，支持系统的功能实现，如用户下单、商品管理、订单处理等。同时，ER 图在数据库设计过程中起到规范作用，确保数据的完整性和一致性，方便后续的数据库实现和优化。

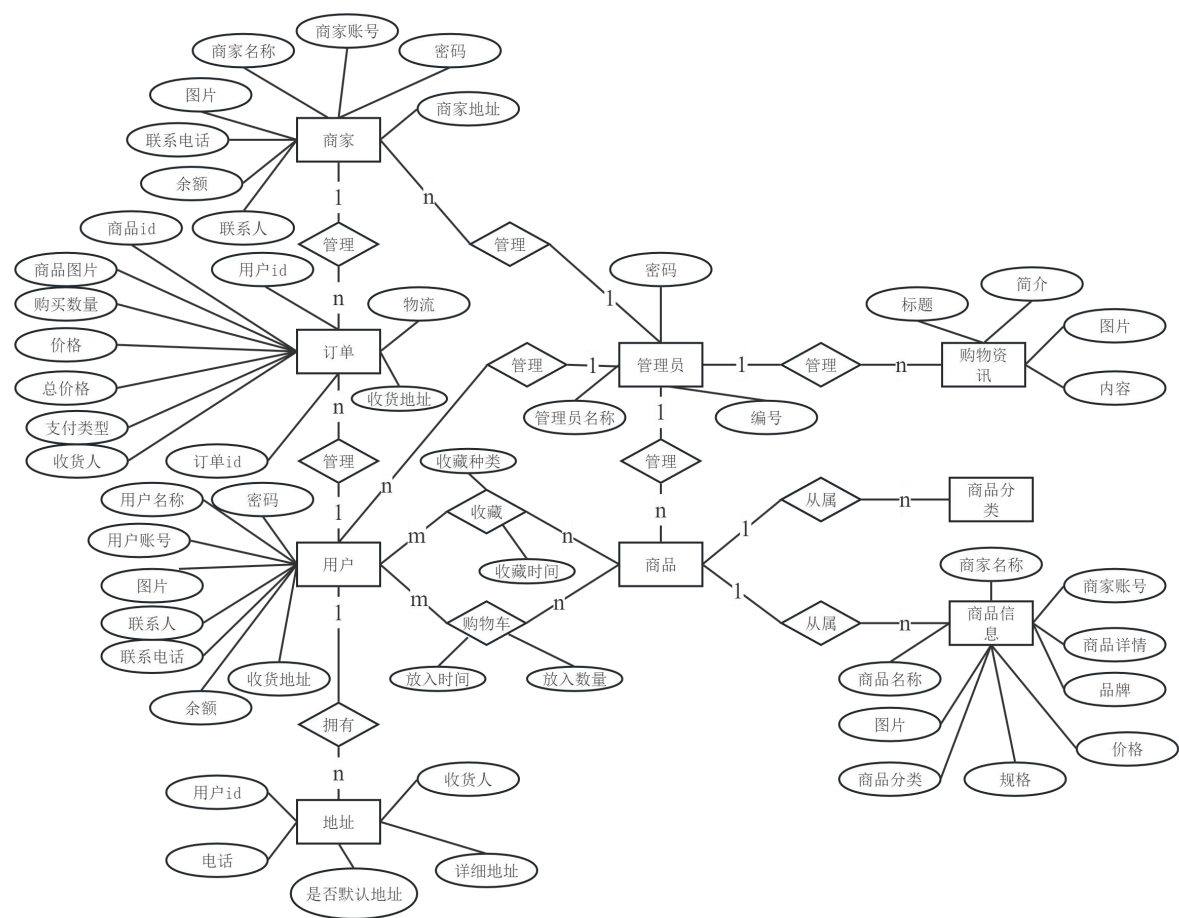


图 3-1 E-R 图

3.3 数据流图

智慧商城的数据流图通过图形化表示展示了系统中各模块之间的数据流转过程。它详细描述了从用户发起请求到系统处理、数据库交互及最终响应的各个步骤，帮助理解系统的整体工作流程。数据流图有效地展示了数据在不同功能模块之间的流动路径，如用户登录、商品浏览、购物车管理、订单处理、支付、评

价等，确保各环节紧密衔接，提高系统的设计和开发效率。

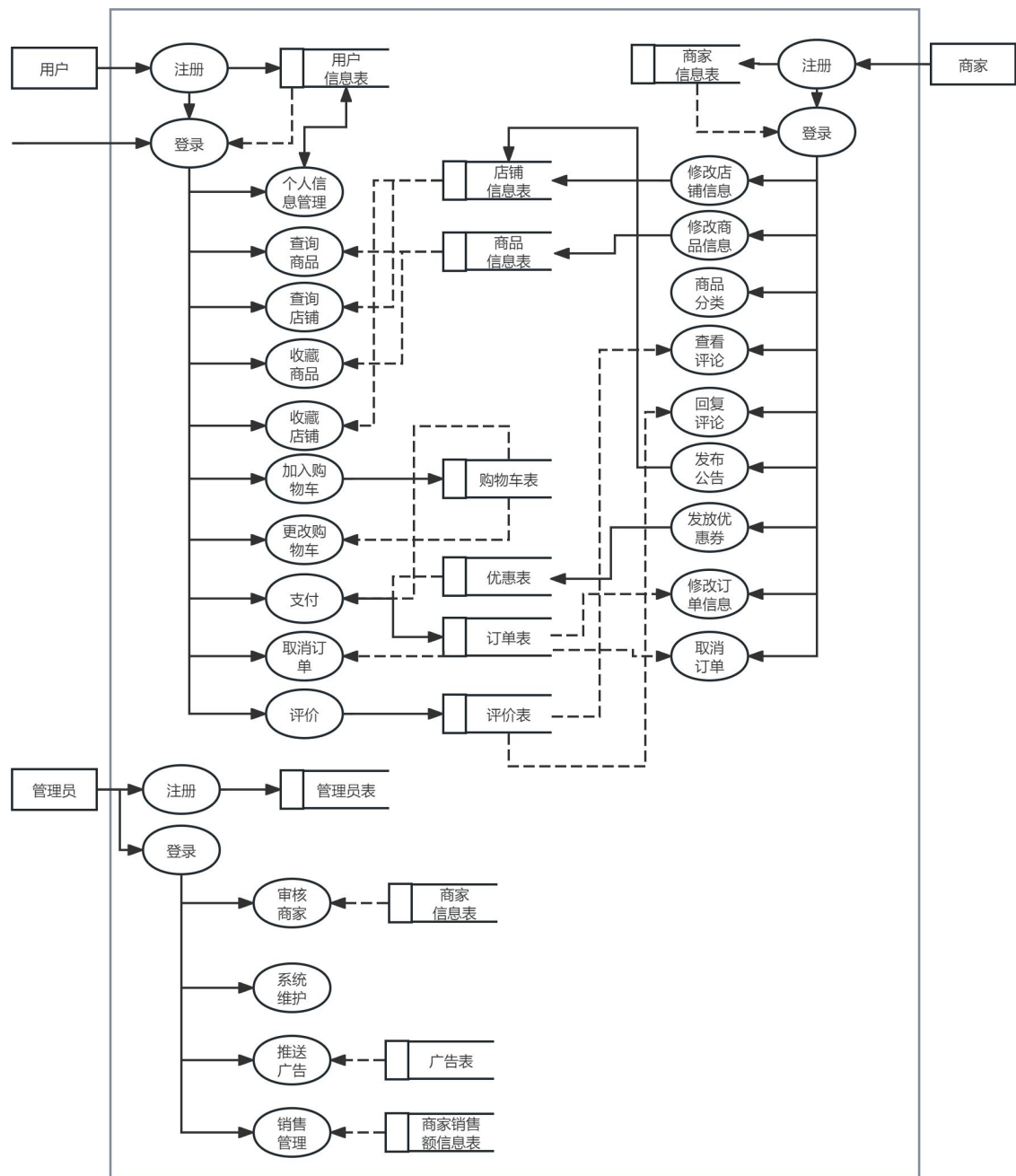


图 3-2 数据流图

第 4 章 系统架构与功能实现

4.1 系统架构设计

智慧商城系统总体架构采用前后端分离的设计原则，主要包括以下几个部分：

小程序端使用微信小程序框架，负责与用户进行交互。它主要处理用户输入，将用户请求发送到后端 API，并将后端返回的数据进行展示。微信小程序框架提供了良好的用户体验，简化了前端开发，提高了开发效率。

视图层基于微信小程序前端框架，实现各个功能模块的视图展示。通过使用 WXML、WXSS 和 JavaScript 等技术，视图层构建了丰富的用户界面，展示数据并处理用户操作，确保用户可以直观、便捷地与系统交互。

业务层主要由 Spring Boot 和 Flask 框架组成，处理系统的核心业务逻辑。Spring Boot 负责实现用户管理、商品管理、订单管理、支付、评价、收藏、推荐、同城购物和直播等功能。Flask 则专注于处理虚拟试穿功能，利用 VisionKit 和 VOfitFusion 模型实现虚拟试鞋和试衣功能。业务层的设计确保了系统功能的完整性和高效性。

数据层使用 MyBatis 持久层框架负责数据库操作，进行数据的增删改查。MyBatis 提供了数据访问接口，简化了数据库操作，提高了数据处理效率，确保了系统数据的完整性和一致性。

数据库层包括 MySQL 和 Redis。MySQL 用于存储核心数据，包括用户信息、商品信息、订单信息、评价、收藏等。Redis 用于缓存数据，存储临时数据以提高系统响应速度，缓解数据库压力。两者的结合保证了数据存储的可靠性和系统性能的优化。

通过这种分层设计，智慧商城系统能够实现各个模块的独立开发和维护，提高系统的稳定性和扩展性，确保用户能够享受到流畅的使用体验。系统架构图如下。

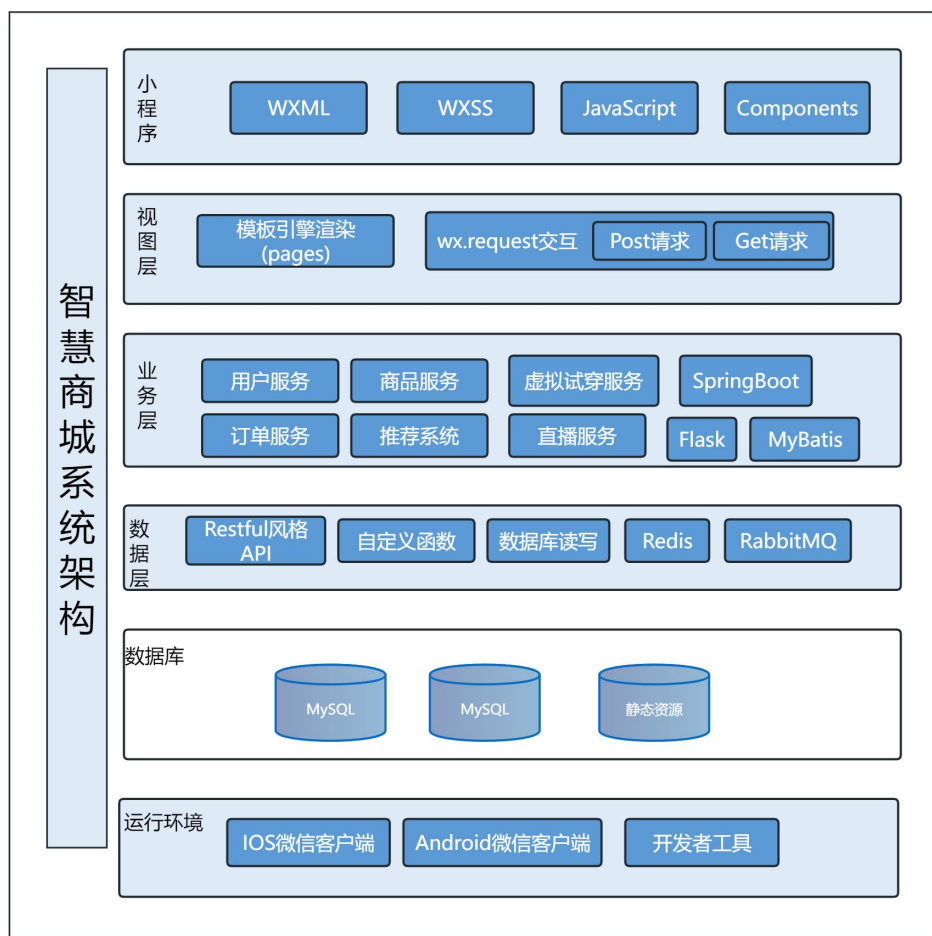


图 4-1 系统架构图

4.2 功能模块设计

用户功能设计：

智慧商城系统为用户提供了丰富且便捷的功能，以提升整体购物体验。用户可以通过微信授权登录快速进入系统，进行注册或退出登录。首页展示用户上传的商品帖子，用户可以浏览和筛选商品，按照上架时间、价格、销量等进行排序，也可以根据大类或小类进行筛选。用户可以管理收货地址，添加、删除和修改地址信息。购买过程中，用户可以查看商品详情，选择规格，选择配送地址并提交订单，使用虚拟支付完成交易。

系统还提供了个人中心功能，用户可以修改密码、姓名等信息；订单页面展示了所有订单，包括未支付、待发货和待收货的订单。用户可以对已收货商品进行评价，查看和管理收藏的商品，并将商品加入购物车。此外，系统基于协同过

滤算法和 ResNet 图像检索算法，为用户提供商品推荐服务。用户还可以体验虚拟试鞋和试衣功能，前者通过 VisionKit 实现，后者通过 VOfitFusion 模型实现。

为增强用户体验，系统支持同城购物功能，用户可以浏览同城商品，选择快递配送或到店自取，促进实体店经济和本地配送服务的发展。系统还集成了 WebSocket，实现了客服聊天功能，用户可以随时与客服进行实时沟通，解决购物过程中的问题。

商家功能设计：

智慧商城系统为商家提供了全面的商品和订单管理功能，帮助商家高效运营。商家可以通过系统管理自己的商品信息，包括添加、删除和修改商品，确保商品信息的准确性和更新。订单管理功能使商家能够查看和处理用户的订单，管理订单的发货状态，保证及时交付商品。系统还支持商家发起直播，利用直播功能进行商品展示和推广，吸引更多的用户关注和购买。同时，商家可以查看用户对其商品的评价，了解用户反馈，进行改进和优化。通过这些功能，商家能够更加便捷地管理业务，提升用户满意度，促进销售增长。

管理员功能设计：

智慧商城系统中的管理员角色负责管理整个系统的运营和维护。管理员拥有全面的管理权限，包括用户信息管理、商家信息管理和首页帖子管理。用户信息管理模块允许管理员查看和管理所有用户的账户信息，确保用户信息的准确性和安全性。商家信息管理模块使管理员能够审核和管理商家的注册信息和商品发布信息，保证平台上商家的合法性和商品的合规性。首页帖子管理模块让管理员能够审核用户发布的商品帖子，删除不符合规定的内容，维护平台的内容质量。管理员还可以管理商品评价，对用户提交的商品评价进行审核和处理，确保评价内容的合法性和真实性。此外，管理员能够查看和处理所有订单，监督订单的状态和处理进度。

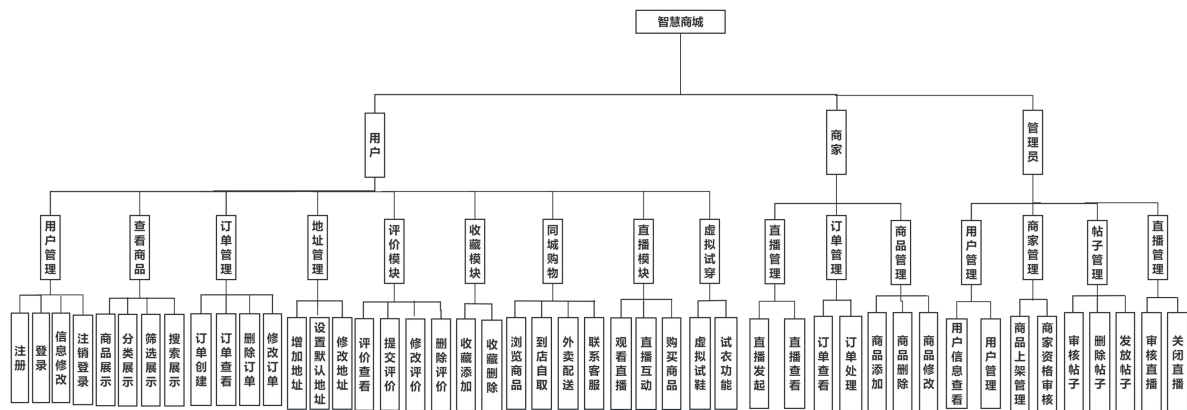


图 4-2 功能模块图

4.3 系统主要功能实现

4.3.1 WebSocket 实现客服聊天

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议。它通过在握手阶段将 HTTP 升级协议为 WebSocket 协议来实现。在握手成功后，客户端和服务端可以互相推送消息，而不需要不断的发起 HTTP 请求和响应。

Java 实现 WebSocket 长连接代码：

```
@Slf4j
@ServerEndpoint(value = "at/{username}/{followName}")
@Component
public class WebSocket {
    private static Map<String,WebSocket> onLineUsers = new ConcurrentHashMap<>();
    private Session session;
    private String username;
    private String followName;
    private static final ObjectMapper MAPPER = new ObjectMapper();
    private MessageService messageService =
        SpringContextUtils.getBean(MessageService.class);
    @OnOpen
    public void onOpen(Session session, @PathParam(value="username")
String username, @PathParam(value="followName")String followName){
        .username = username;
        this.followName = followName;
        this.session = session;
        onLineUsers.put(username+followName, this);
    }
}
```

```

private Set<String> getNames(){
    return onLineUsers.keySet();
}
@OnMessage
public void onMessage(String message, Session session){
    try{
        Packet msg = MAPPER.readValue(message, Packet.class);
        Message chat = new Message();
        //发送人
        chat.setSender(msg.getFrom());
        //接收人
        chat.setReceiver(msg.getToWho());
        //消息
        chat.setMsg(msg.getMessage());
        //时间
        chat.setTime(LocalDateTime.now().toString());
        //判断接收方是否在线,发送消息
        if(onLineUsers.get(msg.getToWho() + Integer.toString(msg.getFrom())) != null){
            onLineUsers.get(msg.getToWho() + Integer.toString
                (msg.getFrom())).session.getBasicRemote().sendText(message);
            chat.setFlag(1);
            messageService.insertMessage(chat);
        }else{
            chat.setFlag(0);
            messageService.insertMessage(chat);
        }
    }catch (Exception e){
        e.printStackTrace();
    }
}
@OnClose
public void onClose(Session session){
    if (username+followName!= null){
        onLineUsers.remove(username+followName);
    }
}

```

上面代码中，`@ServerEndpoint(value = "at/{username}/{followName}")` 注解用于标识当前类是一个 WebSocket 的处理类。`@OnOpen`、`@OnMessage`、`@OnClose` 注解分别对应 WebSocket 的事件，在相应的事件发生时会被触发执行相应的方法。

`onMessage` 方法：当服务器收到一条消息时，它会解析收到的消息，将其转

换为 Packet 对象，并创建一个 Message 对象来保存消息的详细信息。然后，它会检查接收方是否在线，如果在线则发送消息并标记为已发送，否则标记为未发送，并将消息保存到数据库中。如果处理过程中发生异常，异常信息会被打印出来。

微信小程序接入 WebSocket 长连接：

```
SocketInit(){
  wx.connectSocket({
    url: "ws://localhost:8080/chat/"+userid+'/'+this.data.recordId,
    forceCellularNetwork: true,
    perMessageDeflate: true,
    protocols: [],
    tcpNoDelay: true,
    timeout: 0,
    success: (res) => {},
    fail: (res) => {},
    complete: (res) => {},
  })
  wx.onSocketClose((result) => {

  })
  wx.onSocketMessage((result) => {
    data = result.data
    packet = JSON.parse(data)
    msg = {sender: packet.from,
      text: packet.message,
      flag:1,
      time:this.data.time}
    newList = this.data.chatList
    newList.push(msg)
    this.setData({
      chatList:newList
    })
  })
}
```

当连接成功后，它会监听 WebSocket 消息事件，并将接收到的消息解析为 JSON 对象，更新页面的聊天记录。连接关闭事件的处理函数为空，仅用于记录连接关闭情况。通过调用 wx.connectSocket 建立连接，并使用 wx.onSocketMessage 处理实时接收的消息，将其添加到当前的聊天记录中并更新页面显示。

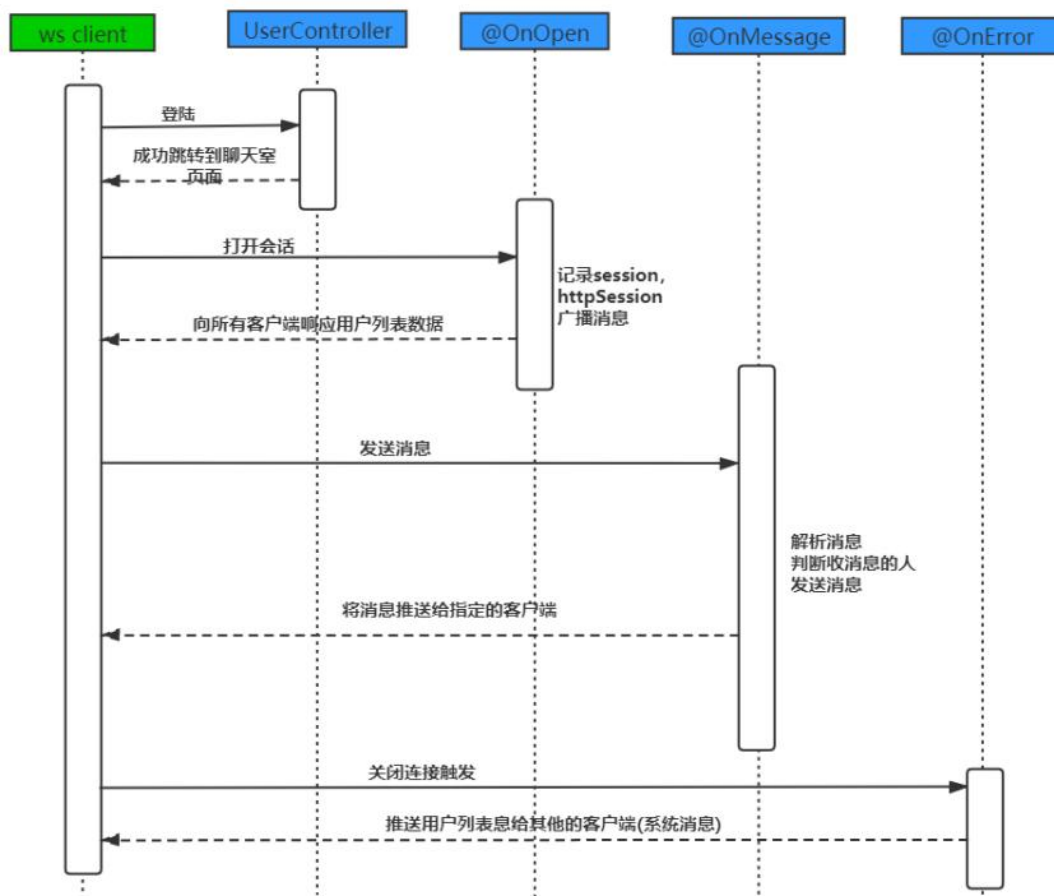


图 4-3 Websocket 时序图

1. 第一个页面：消息记录列表页面（pages/index/index.js）

页面功能：

在 onLoad 生命周期中，获取全局的用户信息和用户 ID，并将其设置到页面的 data 中。在 onShow 生命周期中，设置页面的 userInfo 和 records，并调用 getRecords 方法获取消息记录。使用 setInterval 定时器每隔 3 秒钟调用一次 getRecords 方法，以更新消息记录列表。

数据处理：

getRecords 方法通过 wx.request 发送 GET 请求获取消息记录列表，并将返回的数据设置到页面的 records 变量中。

事件处理：

startChat 方法用于处理点击事件，获取选定记录的索引，并通过 wx.navigateTo 跳转到聊天页面，并将对应的接收者 ID 作为参数传递。

第二个页面：聊天页面 (pages/message/chat.js)

数据处理：

getChatList 方法通过 wx.request 发送 GET 请求获取与指定用户的聊天记录，并将返回的数据设置到页面的 chatList 变量中。

getInfo 方法通过 wx.request 获取用户信息，分别获取当前用户和接收者的头像，并将其设置到页面数据中。

WebSocket 设置：

SocketInit 方法使用 wx.connectSocket 方法连接 WebSocket，配置连接的 URL，使用用户 ID 和聊天记录 ID 作为连接路径的一部分。

在 onSocketMessage 方法中处理收到的 WebSocket 消息，解析并添加到页面的 chatList 中，实现实时聊天的功能。

事件处理：

publishMessage 方法用于发送聊天消息，首先验证消息内容不为空，然后通过 wx.sendSocketMessage 方法发送消息内容，并将发送的消息添加到页面的 chatList 中。handleInput 方法用于处理输入框的输入事件，设置输入内容到页面的 inputValue 中。



图 4-4 点击客服聊天



图 4-5 聊天页面



图 4-6 消息列表界面

4.3.2 商品购买

时序图展示了系统中各个组件或模块之间的交互过程及其时间顺序，通过垂直时间轴上的消息传递来描述系统行为。它有助于理解系统在执行某一特定任务时各个部分的协作方式和调用顺序，从而发现潜在的逻辑问题和优化点。通过时序图，可以清晰地看到用户、前端、后端以及数据库等各部分在处理请求时的详细交互步骤，为开发和调试提供直观的参考。商品购买流程的时序图如下：

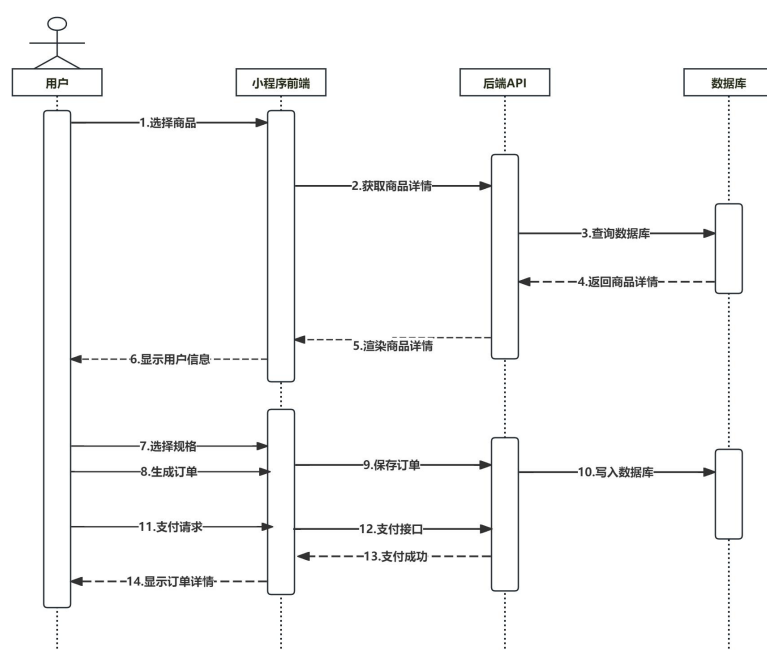


图 4-7 商品购买的时序图

前端页面逻辑：

1. 全局变量引入与数据初始化

`app.globalData.baseUrl` 是小程序的全局数据，用于获取接口的基础 URL。页面通过 `Page` 构造函数绑定了 `data` 对象，包括：`selectedAddress`：选中的收货地址，默认为 `null`。

`orderItems`：订单商品信息，暂时注释掉了初始化的示例数据。

`goodid`、`goodname`、`goodimg`、`standardname`、`goodprice`、`goodshopid`、`goodshopname`、`touxiang`：分别存储商品的相关信息，初始值为 `app.globalData.baseUrl`。

2. 页面加载时执行的操作（`onLoad` 方法）

获取选中的地址信息（`addressId`），如果存在则通过 `wx.request` 请求获取

该地址的具体信息，并更新页面的 `selectedAddress`。

获取商品信息 (`goodsId`)，同样使用 `wx.request` 请求获取商品的详细信息，并更新页面相关数据，如 `goodid`、`goodname`、`goodimg`、`goodshopid`。

获取商品规格信息 (`standardId`)，通过 `wx.request` 请求获取商品规格的详细信息，并更新页面的 `goodprice`、`standardname`。

获取用户信息 (`userid`)，通过 `wx.request` 请求获取用户的详细信息，主要获取用户头像 `touxiang`。

打印输出 `baseUrl`，用于调试时确认接口的基础 URL 是否正确。

3. 页面展示时执行的操作 (`onShow` 方法)

当页面重新展示时，重新调用 `onLoad` 方法，实现页面数据的刷新和更新。
提交订单逻辑 (`createOrder` 方法)

4. 当用户点击提交订单按钮时，触发 `createOrder` 方法。

构造订单数据 (`orderData`) 包括 `userid`、`goodid`、`standardid`、`addressid`。
使用 `wx.request` 向后端发送 `POST` 请求，创建订单。如果创建成功 (`res.data.code == 200`)，则显示成功提示并跳转至商品详情页面。



图 4-8 选择规格



图 4-9 填写地址



图 4-10 提交订单

4.3.3 地址管理

在新增地址页面，用户输入收件人、联系电话和收货地址，当用户点击保存按钮时，会触发 `addAddress` 函数，该函数通过 `wx.request` 发起 POST 请求，将地址信息发送到服务器，成功后跳转回地址列表页。

在地址列表页面，页面加载时会触发 `getUserAddresses` 函数，通过 `wx.request` 发起 GET 请求获取用户的所有地址信息并更新页面数据。用户点击新增按钮，会跳转到新增地址页面。用户点击编辑按钮，会跳转到编辑地址页面，并传递地址 ID。用户点击删除按钮，会触发 `deleteAddress` 函数，通过 `wx.request` 发起 DELETE 请求删除地址信息，成功后刷新地址列表。用户点击地址项会触发 `selectAddress` 函数，选中或取消选中该地址，并将选中的地址 ID 保存到 `globalData` 中。

在编辑地址页面，页面加载时会根据传递的地址 ID 通过 `wx.request` 发起 GET 请求获取该地址的信息并更新页面数据。用户输入新的收件人、联系电话和收货地址，当用户点击保存按钮时，会触发 `saveAddress` 函数，通过 `wx.request` 发起 PUT 请求更新地址信息，成功后跳转回地址列表页。

4.3.4 首页

1. 搜索框和搜索功能：

页面顶部有一个搜索框，显示当前的搜索占位符。

用户点击搜索按钮触发搜索功能。

2. 分类页面跳转：

点击搜索框右侧的分类图标可以跳转到分类页面，具体实现逻辑在 `goToKind` 函数中。

3. 轮播图：

页面顶部包含一个自动播放的轮播图，显示多张图片。轮播图数据存储在 `swiperImages` 数组中，通过 `swiper` 组件实现自动播放和指示点显示。

4. 直播功能跳转：

页面中有一个直播功能入口，点击图片可以跳转到直播页面，具体实现在

gotoLive 函数中。

5. 购买页面跳转：

点击特定图片可以跳转到购买页面，具体实现在 gotoBuy 函数中。

6. 新品和其他推荐入口：

页面底部显示新品、爆款、新课专项和会员福利等推荐入口，点击可以跳转到对应页面，如 gotoNew 函数中跳转到新品列表页面。

7. 页面加载和占位符动画：

页面加载时会调用 onLoad 函数，启动占位符动画功能 startPlaceholderAnimation。占位符动画每 1.5 秒切换一次，展示不同的搜索占位符文本。



图 4-11 商城首页



图 4-12 商品列表

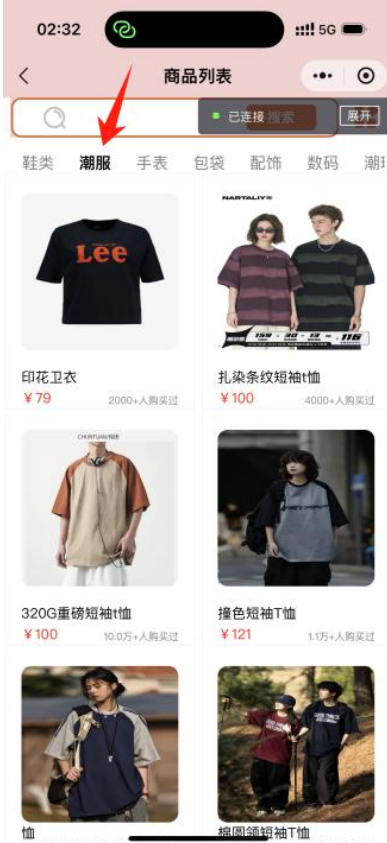


图 4-13 商品列表中的分类

4.3.5 商品列表页

页面加载时，触发 `onLoad` 函数调用 `getCategories` 函数，通过 `wx.request` 获取分类数据，成功后更新 `categories` 并加载第一个分类的商品。

用户点击分类按钮，触发 `selectCategory` 函数，通过 `e.currentTarget.dataset` 获取分类 ID，更新 `selectedCategoryId` 并调用 `getGoodsByCategory` 函数获取该分类下的商品。

用户点击商品，触发 `goToDetail` 函数，通过 `e.currentTarget.dataset.id` 获取商品 ID，跳转到商品详情页面，并携带商品 ID 作为参数。

用户点击搜索框右侧分类图标，触发 `goToKind` 函数，跳转到分类页面。

用户点击搜索框左侧图标，触发 `goToKind` 函数，跳转到分类页面。

商品数量格式化函数 `formatNumber` 根据商品数量大小返回相应格式字符串。

用户点击搜索按钮，触发 `goToSearch` 函数，跳转到搜索页面。

4.3.6 筛选商品

主要方法：

`showFilterPanel`：点击筛选按钮时触发，通过 `setData` 方法更新 `showFilterPanel` 的状态，实现筛选面板的显示与隐藏。

filter.wxml 页面：

结构：`filter-panel`：筛选面板的外层容器。`filter-content`：筛选内容区域。`input`：用于输入最低价和最高价的文本框。

`button`：确定按钮，用于提交筛选条件。

处理流程：用户在 `buy.js` 页面点击筛选按钮，触发 `showFilterPanel` 方法。`showFilterPanel` 方法通过 `setData` 更新 `showFilterPanel` 的状态，控制 `filter.wxml` 页面中 `filter-panel` 的显示与隐藏。用户在 `filter.wxml` 页面输入最低价和最高价，点击确定按钮提交筛选条件。



图 4-14 查看新品



图 4-15 筛选条件

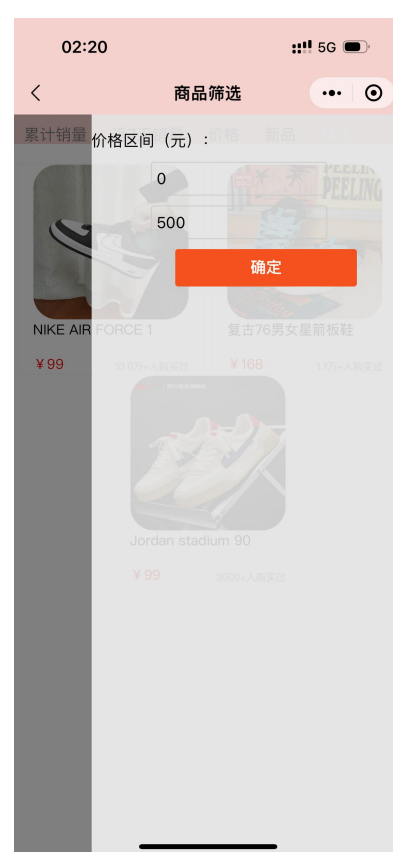


图 4-16 设置区间

4.3.7 商品详情页

1. 页面加载 (onLoad 方法)

1) 用户信息获取:

通过 `wx.request` 请求获取用户信息, 并将结果存储在 `user` 中。

商品信息获取:

通过 `wx.request` 请求获取指定商品的详细信息, 包括图片、描述、销量等, 并进行格式化处理后存储在 `good` 中。

2) 商品最低价格获取:

通过 `wx.request` 请求获取指定商品的最低价格, 并存储在 `minprice` 中。

商品规格获取:

通过 `wx.request` 请求获取指定商品的所有规格信息, 并存储在 `standards` 中。

3) 收藏状态获取:

调用 `checkExist` 方法检查当前用户是否已收藏该商品，并将结果存储在 `isCollected` 中。

4) 评论列表获取:

通过 `wx.request` 请求获取指定商品的评论列表，并将最多两条评论存储在 `comments` 中，同时记录评论总数 `commentsSum`。

5) 页面滚动监听:

在 `onPageScroll` 方法中监听页面滚动事件，并实时更新页面的 `scrollTop` 值，以响应页面滚动操作。

2. 功能方法

1) 收藏状态切换 (`toggleCollect` 方法):

当用户点击收藏按钮时，根据当前 `isCollected` 的状态进行收藏或取消收藏的操作，并通过 `wx.request` 向后端发送收藏或取消收藏的请求。

2) 弹窗显示与隐藏 (`showPopup`、`closePopup` 方法):

`showPopup` 方法用于显示弹窗，并根据 `navigateTo` 的值确定弹窗内容的跳转目标。`closePopup` 方法用于关闭弹窗。

3) 规格选择与确认 (`selectStandard`、`confirmSelection` 方法):

`selectStandard` 方法用于选择商品规格，更新 `selectedStandardId`。
`confirmSelection` 方法在用户确认选择规格后，根据 `navigateTo` 的值执行相应操作，如加入购物车或直接购买。

4) 页面内部跳转 (`toBlock1`、`toBlock2` 等方法):

这些方法用于实现页面内部滚动到指定位置的功能，通过 `wx.pageScrollTo` 方法实现。



图 4-17 商品详情页



图 4-18 商品细节页



图 4-19 展开商品的所有评论

4.3.8 收藏商品

1. 页面加载:

页面加载时, 调用 `fetchFavoriteGoods` 函数, 获取用户收藏的商品列表。

2. 页面显示:

页面每次显示时, 再次调用 `fetchFavoriteGoods` 函数, 刷新用户收藏的商品列表。

3. 获取收藏的商品:

发送 GET 请求到服务器获取用户收藏的商品列表, 将返回的数据设置到页面数据 `goods` 中。

4. 跳转到商品详情页面:

用户点击某个收藏的商品项时, 触发 `goToDetail` 函数, 获取商品 ID, 并跳转到商品详情页面。

4.3.9 虚拟试衣

1. 页面初始化：设置默认衣服图片的 URL。默认的衣服图片 URL 存储在 data 中的 `garmImageUrl`。
 2. 选择和上传用户照片：用户点击“更换”按钮触发 `chooseImage` 方法，选择一张照片。调用 `wx.chooseImage` 方法，用户选择照片后调用 `uploadImage` 方法上传该照片。`uploadImage` 方法将选择的图片上传到后端服务器，并将返回的图片 URL 保存到 data 中的 `vtonImageUrl`。
 3. 生成试穿效果：用户点击“生成试穿效果”按钮触发 `generateImage` 方法。`generateImage` 方法检查是否已上传用户照片，如果未上传，则提示用户上传。调用 `wx.request` 方法向后端发送请求，传递用户照片和衣服图片的 URL，生成试穿效果图。
- 后端返回生成的图片 URL，并将其保存到 data 中的 `imageUrl`。
5. 显示试穿效果：页面显示生成的试穿效果图片和其链接。



图 4-20 虚拟试衣界面



图 4-21 虚拟试衣效果



图 4-22 收藏商品

4.3.10 购物车

页面加载时，触发 `onLoad` 函数调用 `loadCartItems` 函数，通过 `wx.request` 获取购物车商品数据，成功后更新 `cartItems`、`totalPrice` 和 `checkboxStatus`。

用户点击移除按钮，触发 `removeCartItem` 函数，通过 `e.currentTarget.dataset.id` 获取商品 ID，并通过 `wx.request` 发起删除请求，成功后重新加载购物车商品数据。

用户点击商品勾选框，触发 `selectCartItem` 函数，通过 `e.currentTarget.dataset.index` 获取商品索引，重置所有勾选框状态并设置当前商品为勾选状态，同时更新 `totalPrice` 和 `sele_index`。

用户点击结算按钮，触发 `checkout` 函数，通过 `this.data.cartItems[this.data.sele_index]` 获取当前选中的商品信息，并设置全局变量的值后跳转到预订单页面。

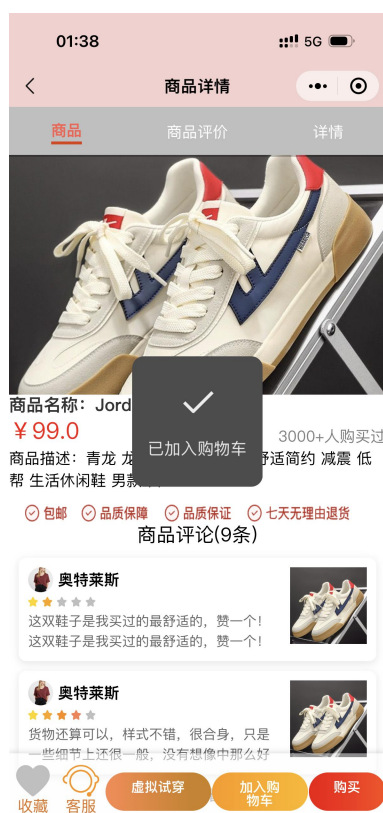


图 4-23 加入购物车成功

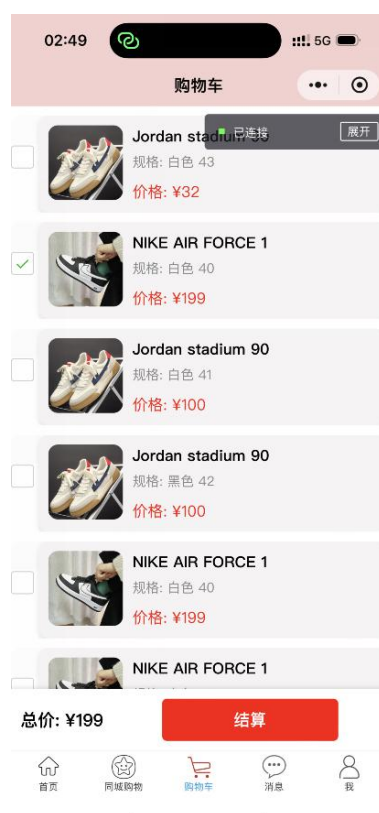


图 4-24 购物车界面

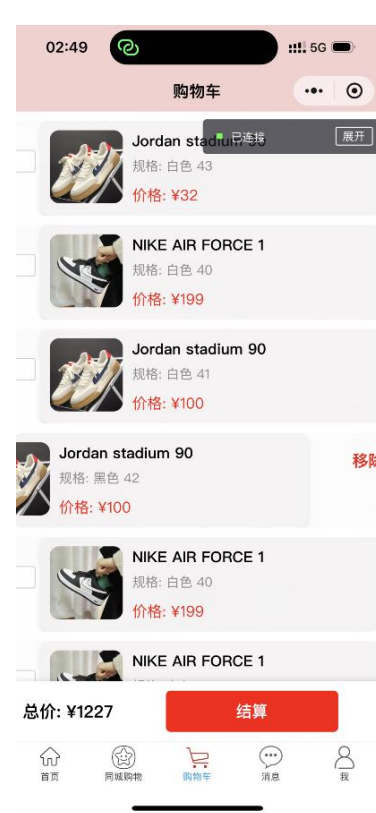


图 4-25 删除购物车中的商品

4.3.11 同城购物

1. 页面初始化和地理位置获取：

在页面加载时（onLoad 函数），通过 `wx.getLocation` 获取用户的地理位置信息（经度和纬度）。成功获取地理位置后，将经纬度存储到页面数据中（`latitude` 和 `longitude`），然后调用 `getCategories` 方法获取分类信息。

2. 获取分类信息：

`getCategories` 方法通过 `wx.request` 发送 GET 请求，调用后台接口获取分类信息，传递当前的地理位置（经纬度）作为参数。

请求成功后，将返回的分类数据存储到页面数据的 `categories` 中。如果分类数据不为空，默认选择第一个分类，并调用 `getProductsByCategory` 方法获取该分类下的商品信息。

3. 获取分类下的商品信息：`getProductsByCategory` 方法通过 `wx.request` 发送 GET 请求，调用后台接口获取指定分类下的商品信息，传递当前选中的分类 ID 和地理位置（经纬度）作为参数。请求成功后，将返回的商品数据存储到页面数据的 `products` 中。

4. 选择分类：用户点击某个分类时，触发 `selectCategory` 方法，获取被点击分类的 ID，并将其设置为当前选中的分类 ID（`selectedCategoryId`）。然后调用 `getProductsByCategory` 方法，重新获取该分类下的商品信息。

5. 跳转到商品详情页面：用户点击某个商品时，触发 `goToDetail` 方法，获取被点击商品的 ID。

使用 `wx.navigateTo` 方法跳转到商品详情页面，并将商品 ID 作为参数传递给详情页面。

6. 导航到店铺位置：用户点击商品中的店铺信息时，触发 `navigateToShop` 方法，获取店铺的经纬度。使用 `wx.openLocation` 方法打开微信内置地图，并导航到店铺的位置。



图 4-26 显示商家距离

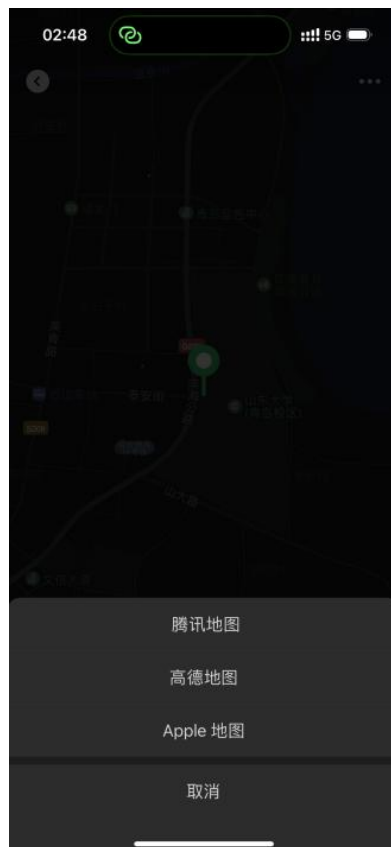


图 4-27 选项



图 4-28 到店方式

4.3.12 商品评价

- 1. 页面加载:** 从 options 获取商品 ID (goodid) 和订单 ID (orderid)。发送请求获取商品信息，并将商品名称和图片设置到页面数据中。
- 2. 评分处理:** 用户点击星星图片触发 score 函数，更新当前选择的星级数。循环遍历星星数组，将选中的星星状态设置为 flag，并更新到页面数据中。
- 3. 输入评论:** 用户在文本框输入评论内容触发 inputComment 函数，更新评论内容到页面数据中。
- 4. 提交评价:** 用户点击提交按钮触发 submitComment 函数，收集评分、评论内容、用户 ID、商品 ID 和订单 ID，构建提交的数据对象。发送请求将评价数据提交到服务器，并根据提交结果显示提示信息。
- 5. 界面显示:** 页面包含商品信息显示（图片和名称）、星级选择控件、评论输入框和提交按钮。用户可以通过点击星星选择评分，通过文本框输入评论内容，并

点击提交按钮提交评价。



图 4-29 待收货的订单



图 4-30 待评价的订单

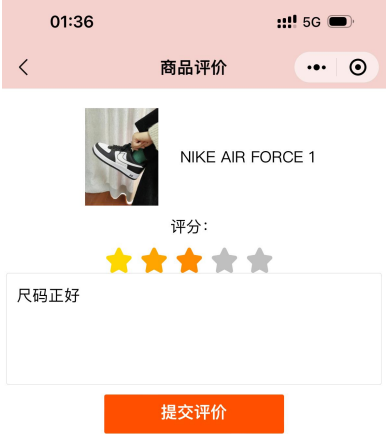


图 4-31 评价界面

4.3.13 订单页面

1. 页面全局变量和数据初始化

`const app = getApp();` 和 `const userid = app.globalData.userid;` 获取小程序实例和全局变量中的用户 ID。

Page 构造函数中使用 data 对象初始化了页面的数据状态：`orders`：存储所有用户订单的原始数据。

`filteredOrders`：根据状态筛选后的订单数据。

`activeStatus`：当前选中的订单状态，默认为 '全部'。

2. 页面加载 (onLoad 方法)

从页面参数 options 中获取传入的 `activeStatus`，默认为空字符串。

调用 `fetchOrders` 方法根据用户 ID 和传入的状态 `activeStatus` 获取订单数

据，并更新页面状态。

3. 页面显示 (onShow 方法)

每次页面显示时调用 `fetchOrders` 方法，以确保状态与数据同步更新。

4. 获取订单数据 (fetchOrders 方法)

使用 `wx.request` 向后端发送 GET 请求，获取指定用户 ID 的订单列表数据。请求成功后，根据返回的订单数据 `res.data.data` 进行过滤：如果 `status` 不是‘全部’，则根据 `status` 筛选出符合条件的订单数据。

使用 `setData` 更新页面的 `orders` 和 `filteredOrders`，实现数据的绑定与展示。

5. 筛选订单 (filterOrders 方法)

当用户点击状态栏中的订单状态项时，触发 `filterOrders` 方法。

根据点击的状态项 `e.currentTarget.dataset.status` 进行订单数据的重新筛选：若状态为‘全部’，则不进行额外的过滤，展示所有订单。否则，根据状态项筛选出符合条件的订单数据，并更新页面的 `filteredOrders` 和 `activeStatus`。

6. 跳转至订单详情页 (goToOrderDetail 方法)

当用户点击某个订单项时，获取订单的 `id`，并使用 `wx.navigateTo` 跳转至订单详情页 `/pages/orderDetail/orderDetail?id=${orderId}`。

7. 跳转至待评价页面 (goToComment 方法)

当用户点击订单状态为‘待评价’的订单项中的评价按钮时，获取商品 ID (`goodId`) 和订单 ID (`orderId`)，并使用 `wx.navigateTo` 跳转至评价页面 `/pages/comment/comment?goodid=${goodId}&orderid=${orderId}`。

4.3.14 个人信息

1. 页面全局变量和数据初始化

`const app = getApp();` 获取小程序实例。`const baseUrl = app.globalData.baseUrl;` 和 `const userid = app.globalData.userid;` 获取全局变量中的基础 URL 和用户 ID。

`Page` 构造函数中使用 `data` 对象初始化了页面的数据状态：`user`：存储用

户的基本信息，包括 name（昵称）、phone（手机号）、description（自我介绍）和 avatar（头像地址）。

imageSrc: 用户头像的显示路径，默认为一个未设置头像的默认图片路径。

relativePath: 用户选择的头像图片的相对路径，默认为 /avatar。

baseUrl: 存储基础 URL，用于拼接完整的接口 URL。

2. 页面加载 (onLoad 方法)

调用 updateAvatar 方法更新用户头像信息。

发起 GET 请求获取当前用户的详细信息，并更新页面的 user 数据。

3. 更新头像 (updateAvatar 方法)

使用 wx.request 发起 GET 请求获取当前用户的详细信息。

根据返回的用户信息，拼接完整的头像 URL，并更新页面的 user 和 imageSrc。

4. 输入事件处理

onNameInput、onPhoneInput、onPasswordInput、onDescriptionInput 方法用于处理用户输入昵称、手机号、密码和自我介绍的事件，更新页面的 user 数据。

5. 选择上传图片 (chooseImage 方法)

调用 wx.chooseImage 接口让用户选择上传的图片。

将选择的图片路径更新至页面的 imageSrc，并调用 uploadImage 方法上传图片。

6. 上传图片 (uploadImage 方法)

校验是否填写了相对路径 relativePath，若未填写则提示用户。

使用 wx.uploadFile 方法上传选中的图片文件至服务器，同时传递相对路径 relativePath。根据上传结果显示上传成功或失败的提示，并更新用户头像信息。

7. 保存修改 (updateUserInfo 方法)

使用 wx.request 发起 PUT 请求更新用户信息至服务器。

根据返回结果显示保存成功或失败的提示，并在保存成功后调用 updateAvatar 方法更新用户头像信息。



图 4-32 个人中心界面

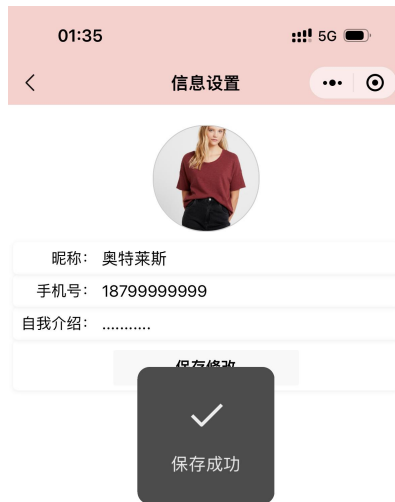


图 4-33 修改个人信息

第 5 章 关键算法和模型

5.1 VisionKit 虚拟试鞋

VisionKit 是一个用于增强应用视觉能力的开发工具包，提供了一系列功能用于处理和分析图像和视频内容。

1. Shoe 鞋部检测

通过摄像头实时检测，可以通过配置决定是否每帧获取用于对鞋子的腿部区域进行蒙版过滤的腿部遮挡纹理。

2. 摄像头实时检测

首先创建 VKSession 的配置，然后通过 VKSession.start 启动 VKSession 实

例。运行过程中，算法实时检测相机中的鞋子，通过 VKSession.on 实时输出鞋子矩阵信息（位置旋转缩放）、8 个关键点坐标。

最后实时获取 VKSession 的帧数 VKFrame，获得 VKCamera。得到 VKCamera.viewMatrix 与 VKCamera.getProjectionMatrix 并结合，鞋子矩阵与关键点信息进行具体渲染。

开启 VKSession 的代码：

```
// VKSession 配置 const session = wx.createVKSession({
  track: {
    shoe: {
      mode: 1 // 1 使用摄像头
    }
  }
})

// 摄像头实时检测模式下，监测到鞋子时，updateAnchors 事件会连续触发
// （每帧触发一次）

session.on('updateAnchors', anchors => {
  // 目前版本能识别 1-2 双鞋子，anchors 长度为 0-2
  anchors.forEach(anchor => {
    // shoedirec 鞋子左右，0 为左，1 为右。
    console.log('anchor.shoedirec', anchor.shoedirec)
    // transform 鞋子的矩阵信息（位置旋转缩放），4*4 行主序矩阵，
    console.log('anchor.transform', anchor.transform)
    // points3d 位置，8 个关键点坐标
    console.log('anchor.points3d', anchor.points3d)
  })
})

// 摄像头实时检测模式下，当鞋子从相机中丢失时，会不断触发
session.on('removeAnchors', () => {
  console.log('removeAnchors')
})
```

```
// 需要调用一次 start 以启动
session.start(errno => {
    if (errno) {
        // 如果失败，将返回 errno
    } else {
        // 否则，返回 null，表示成功
    }})
```

3. 渲染试鞋代码:

```
// 可以理解每一帧都需要执行的行为

// 1. 通过 VKSession 实例的 `getVKFrame` 方法可以获取到帧对象 const
frame = this.session.getVKFrame(canvas.width, canvas.height)

// 2. 获取 VKCamera const VKCamera = frame.camera

// 3. 获取 VKCamera 的 视图矩阵与投影矩阵 const viewMat =
VKCamera.viewMatrix; const projMat =
VKCamera.getProjectionMatrix(near, far);

// 4. 结合 updateAnchors 里面返回 anchor 的 transform、points3d 进
行具体渲染。
```

4. 开启腿部遮挡

摄像头实时模式下，想要开启腿部分割能力。需要保证在 VKSession.start 接口调用，开启 VKSession 后。调用 VKSession 的 updateMaskMode 更新是否开启腿部分割纹理的返回。然后通过 VKFrame 的 getLegSegmentBuffer 获取具体的腿部遮挡纹理 buffer，然后基于 buffer 创建纹理，进行对应的遮挡剔除。

代码:

```
// 1. 开启腿部遮挡纹理获取开关// VKSession.start 后，可以通过
updateMaskMode 更新是否能够获取腿部遮挡纹理
bufferthis.session.updateMaskMode({

    useMask: true // 开启或关闭腿部遮挡纹理 buffer (客户端 8.0.43 默
认开启获取，可以手动关闭)}}
```



```
// 2. 通过 VKSession 实例的 `getVKFrame` 方法可以获取到帧对象 const  
frame = this.session.getVKFrame(canvas.width, canvas.height)  
  
// 3. 通过帧对象, 获取具体腿部分割纹理 Buffer (160*160、单通道) const  
legSegmentBuffer = frame.getLegSegmentBuffer();
```

5. 输出说明

anchor 信息

```
struct anchor{  
    transform, // 鞋子矩阵信息, 位置旋转缩放  
    points3d, // 鞋子关键点数组  
    shoedirec, // 鞋子左右脚, 0 为左脚, 1 为右脚  
}
```

(1) 鞋子矩阵 **transform**

长度为 16 的数组, 表示 行主序 的 4*4 矩阵。

(2) 鞋子关键点 **points3d**

长度为 8 的数组, 表示 鞋子识别的 8 个关键点:

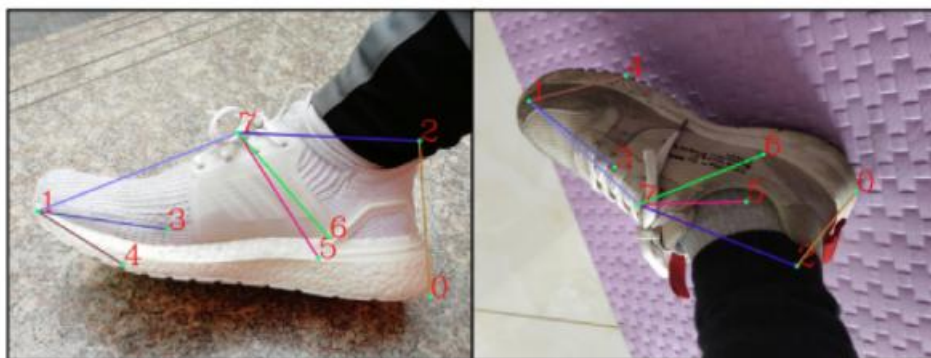
```
Array<Point>(8) point3d
```

每个数组元素结构为:

```
struct Point { x, y, z }
```

表示在鞋子矩阵作用后, 每个点的三维偏移位置。

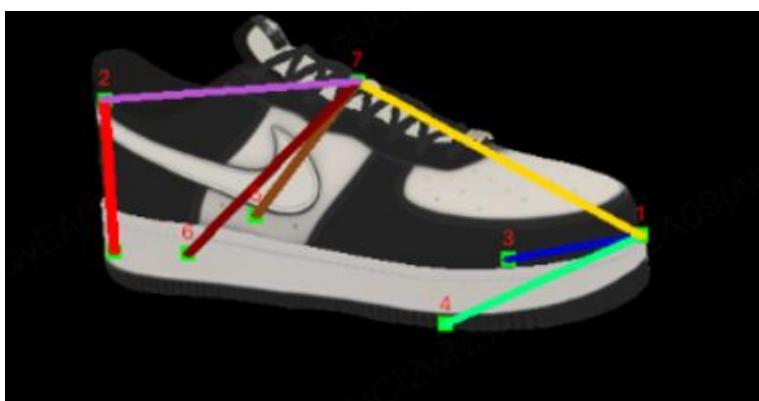
以下为关键点图示与点位解释:



- 点位 0 位于鞋的底部后部，脚后跟处，靠近地面的位置。
- 点位 1 位于鞋子最前面，在竖直方向上大致处于鞋子上表面和鞋子下底面的中间。
- 点位 2 位于鞋的后部，在点位 0 的上方，位于靠近脚踝的位置。
- 点位 3、4 位于鞋子侧面，其中 3 点位于左鞋的左侧，右鞋的右侧。4 点位于左鞋子的右侧，右鞋的左侧。上图中的 3 点处于被遮挡的位置。3，4 点均紧靠鞋子下底面。
- 点位 5，6 位于鞋子侧面，相对于 3，4 更靠后，在鞋子下底面偏上一点的位置。
- 点位 7 大致位于鞋舌的中央。

(3) 腿部分割纹理 Buffer

160*160 的 单通道浮点数颜色 ArrayBuffer，数值 0.0 对应非腿部区域，大于 0.0 代表是腿部区域，越靠近 1.0 越接近是腿部区域。



6. 基于返回信息摆放模型

首先，可以将添加一个节点同步鞋子矩阵信息。再往该节点添加模型，基于鞋子关键点，设置对应的偏移，然后可以缩放模型适配比例，使模型贴合鞋子关键点，达到目标效果。



图 39 AR 试鞋前



图 40 AR 试鞋后

5.2 VOfitFusion(改进的-VTON 模型)

基于图像的虚拟试穿（VTON）是一项流行且前景广阔的图像合成技术，能够显著提升消费者的购物体验并减少服装商家的广告成本。VTON 任务旨在生成目标人物穿戴给定服装的图像。过去几年，众多研究人员付出了巨大努力以获得更自然、准确的虚拟试穿结果。

5.2.1 模型概述

当前基于图像的 VTON 面临两个主要挑战。首先，生成的图像需要足够逼真和自然，以避免失真。最近的大多数虚拟试穿研究都利用生成对抗网络（GAN）或潜扩散模型（LDM）进行图像生成。先前基于 GAN 的方法通常难以生成正确的

服装褶皱、自然光影或逼真的人体。因此，更多的最新研究倾向于使用 LDM 方法，这些方法有效地提高了穿戴图像的逼真度。第二个关键挑战是如何尽可能多地保留服装的细节特征，例如复杂的文本、纹理、颜色、图案和线条等。先前的研究通常执行显式扭曲过程以将服装特征与目标人体对齐，然后将扭曲后的服装输入生成模型（如 GAN 和 LDM）。因此，这些 VTON 方法的性能极度依赖于独立扭曲过程的有效性，而这一过程容易过拟合训练数据。另一方面，一些基于 LDM 的方法尝试通过 CLIP 文本反演学习服装特征，但未能保留细粒度的服装细节。

受到上述基于图像的 VTON 的前景和挑战的启发，本文提出了一种新颖的基于 LDM 的虚拟试穿方法，即 VOfitFusion。首先，本文充分利用预训练潜扩散模型的优势，以确保生成图像的高逼真度和自然的试穿效果，并设计了一个融合的 UNet 以在潜在空间中一步学习服装的细节特征。然后，本文提出了一种融合过程，将服装特征精确对齐到去噪 UNet 的自注意力层中的噪声人体上。通过这种方式，服装特征能够平滑地适应各种目标人体类型和姿态，而不会因为独立的扭曲过程而丢失信息或特征失真。此外，本文还进行了融合掉落操作，在训练中随机丢弃部分服装潜在特征，以实现无分类器指导，从而简单地通过指导比例调整生成结果中的服装控制强度，这进一步增强了本文 VTON 方法的可控性。本文在两个广泛使用的高分辨率基准数据集（即 VITON-HD 和 Dress Code）上训练了本文的 VOfitFusion。广泛的定性和定量评估表明，本文的方法在各种目标人物和服装图像上，在真实感和可控性方面均优于最新的 VTON 方法。

1. Stable Diffusion

本文的 VOfitFusion 是 Stable Diffusion 的扩展，Stable Diffusion 是最常用的潜扩散模型之一。Stable Diffusion 使用了一个变分自编码器（VAE），它由一个编码器 E 和一个解码器 D 组成，以实现图像在潜在空间中的表示。UNet ϵ_θ 被训练来对具有由 CLIP 文本编码器 τ_θ 编码的条件输入的高斯噪声 ϵ 进行去噪。给定图像 x 和文本提示 y ，去噪 UNet ϵ_θ 的训练通过最小化以下损失函数来进行：

$$L_{\text{LDM}} = E_{E(x), y, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(y))\|_2^2]$$

其中 $t \in \{1, \dots, T\}$ 表示前向扩散过程的时间步， z_t 是添加了高斯噪声 $\epsilon \sim \mathcal{N}(0,1)$ 的编码图像 $E(x)$ （即噪声潜在变量）。注意，条件输入 $\tau_\theta(y)$ 通过交叉注意机制与去噪 UNet 相关联。

2. 本文的 V0fitFusion 模型概述。在左侧，服装图像被编码到潜在空间，并输入到融合 UNet 进行一步处理。结合由 CLIP 编码器生成的辅助条件输入，服装特征通过融合技术被纳入去噪 UNet。在训练中特别对服装潜在特征执行融合掉落以实现无分类器引导。在右侧，输入的人体图像根据目标区域进行掩码，并与高斯噪声连接作为去噪 UNet 的输入进行多步采样。去噪后，特征图被解码回图像空间，作为本文的试穿结果。

公式：

$$L_{\text{LDM}} = E_{E(x), y, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(y))\|_2^2]$$

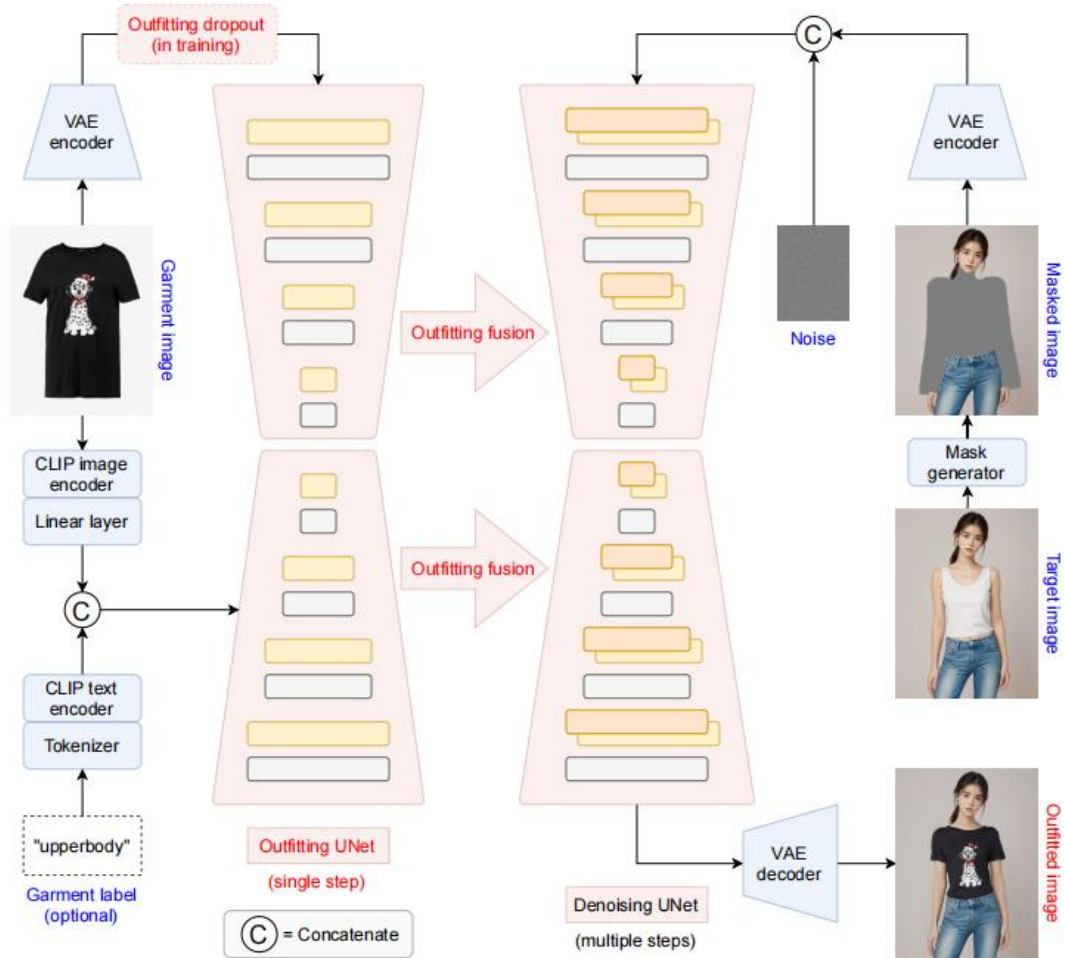


图 5-3 原理图

5.2.2 实现方法

本文的 VOfitFusion 方法包括两个主要组件：融合 UNet 和融合掉落。本文将分别介绍这两个组件的详细设计和实现。

1. 融合 UNet

融合 UNet 的设计基于经典的 UNet 架构，通过在去噪 UNet 的自注意力层中融合服装特征和目标人体特征，实现服装特征的精确对齐。具体来说，本文首先使用 VAE 编码器将目标人体图像 x 和服装图像 g 编码到潜在空间，生成潜在变量 $E(x)$ 和 $E(g)$ 。然后，将这些潜在变量输入到融合 UNet 中，进行进一步处理。

在融合 UNet 中，本文引入了融合机制，通过交叉注意机制，将服装特征与目标人体特征融合在一起。具体实现如下：

编码阶段：将目标人体图像和服装图像分别编码到潜在空间，生成 $E(x)$ 和 $E(g)$ 。

融合阶段：在去噪 UNet 的自注意力层中，通过交叉注意机制，将 $E(g)$ 与 $E(x)$ 融合，实现服装特征的精确对齐。

解码阶段：将融合后的潜在变量通过解码器解码回图像空间，生成最终的穿戴图像。

2. 融合掉落

为了进一步增强本文方法的可控性，在训练过程中引入了融合掉落技术。具体来说，在训练中随机丢弃部分服装潜在特征，以实现无分类器指导，从而简单地通过指导比例调整生成结果中的服装控制强度。这种方法有效地增强了本文 VTON 方法的可控性，使其能够生成不同风格和强度的穿戴图像。

5.2.3 训练策略

本文的 VOfitFusion 通过在两个广泛使用的高分辨率基准数据集（即 VITON-HD 和 Dress Code）上进行训练和评估。具体训练过程如下：

1. 数据预处理：对 VITON-HD 和 Dress Code 数据集中的图像进行预处理，包括图像裁剪、缩放等操作。

2. VAE 编码：使用 VAE 编码器将预处理后的图像编码到潜在空间，生成潜在

变量。

3. 融合 UNet 训练：使用融合 UNet 对潜在变量进行一步处理，通过最小化去噪过程的损失函数进行训练。

4. 融合掉落训练：在训练过程中引入融合掉落技术，随机丢弃部分服装潜在特征，增强模型的可控性。

5.2.4 实验

为了验证本文提出的 VOfitFusion 方法的有效性，本文在 VITON-HD 和 Dress Code 数据集上进行了全面的实验评估。本文的实验设置和结果如下所述。

1. 实验设置

本文在两个高分辨率基准数据集上对 VOfitFusion 进行训练和评估：

VITON-HD：该数据集包含高分辨率的上身服装图像和目标人体图像。

Dress Code：该数据集包含高分辨率的上身服装、下身服装和连衣裙图像，以及目标人体图像。

本文使用标准的评价指标，包括结构相似性指数（SSIM）、峰值信噪比（PSNR）和感知距离（LPIPS），以定量评估生成图像的质量。

2. 实验结果

本文的方法在各种目标人物和服装图像上均表现优异。具体而言，VOfitFusion 在 VITON-HD 和 Dress Code 数据集上的 SSIM、PSNR 和 LPIPS 指标均优于现有的 VTON 方法。此外，研究结果也表明，VOfitFusion 生成的图像在真实感和可控性方面得到了用户的高度认可。

表 5-1 模型评估指标对比

Train/Test Method	VITON-HD/Dress Code				Dress Code/VITON-HD			
	LPIPS ↓	SSIM ↑	FID ↓	KID ↓	LPIPS ↓	SSIM ↑	FID ↓	KID ↓
VITON-HD* [6]	0.187	0.853	44.26	28.82	-	-	-	-
HR-VITON* [27]	0.108	0.909	19.97	7.35	-	-	-	-
LaDI-VTON [32]	0.154	0.908	14.58	3.59	<u>0.235</u>	<u>0.812</u>	<u>29.66</u>	<u>20.58</u>
GP-VTON [52]	0.291	0.820	74.36	80.49	0.266	0.811	52.69	49.14
StableVITON [24]	<u>0.065</u>	<u>0.914</u>	<u>13.18</u>	<u>2.26</u>	-	-	-	-
OOTDiffusion (Ours)	0.061	0.915	11.96	1.21	0.123	0.839	11.22	2.72



图 42 各模型生成图像对比

第 6 章 项目实施与使用

6.1 项目实施

开发工具：

IDE：IntelliJ IDEA 用于后端开发，微信开发者工具用于前端开发

版本控制：Git 和 GitHub 用于代码管理和协作

包管理工具：npm 和 Maven 分别用于管理前端和后端依赖

开发环境：

操作系统：Ubuntu 20.04 LTS 用于服务器部署，Windows 11 用于本地开发

前端技术：微信小程序框架、HTML5、CSS3、JavaScript

后端技术：Spring Boot 用于业务逻辑处理，Flask 用于虚拟试穿功能

数据库：MySQL 用于存储核心数据，Redis 用于缓存

中间件：RabbitMQ 用于订单异步处理

项目部署与配置

1. 准备服务器：

确保服务器上安装了 Java、Python、MySQL 和 Redis。

配置防火墙，开放必要的端口，如 HTTP(S)、MySQL 和 Redis 端口。

2. 部署后端服务：

将 Spring Boot 项目打包为 JAR 文件，并上传到服务器。

在服务器上运行 `java -jar` 命令启动 Spring Boot 服务。

将 Flask 项目打包，并使用 Gunicorn 等 WSGI 服务器进行部署。

3. 配置数据库：

在 MySQL 中创建所需数据库和表结构。

导入初始数据，包括用户、商品和订单等基础数据。

4. 部署前端小程序：

使用微信开发者工具，将小程序代码上传并发布。

确保小程序的 API 地址正确指向已部署的后端服务。

5. 配置中间件：

安装并配置 RabbitMQ，用于处理订单异步消息队列。

6.2 系统使用手册

1. 用户操作说明：

(1) 注册与登录：

用户可以通过微信授权登录进入系统。

完成注册后，可以进行个人信息的修改和管理。

(2) 商品浏览与购买：

用户可以浏览首页推荐商品，进行商品筛选和排序。

点击商品进入详情页，选择规格和数量，加入购物车或直接购买。
选择收货地址并进行虚拟支付，完成订单。

(3) 同城购物：

浏览同城商品，选择快递配送或到店自取。
确认订单并选择配送方式，完成支付。

(4) 虚拟试鞋与试衣：

使用虚拟试鞋功能，通过 AR 技术查看试穿效果。
使用虚拟试衣功能，通过上传照片试穿不同衣物。

(5) 客服聊天：

通过实时客服聊天功能，与客服进行在线沟通。
解决购物过程中遇到的问题。

(6) 直播功能：

观看商家直播，了解商品详情和优惠信息。
参与直播互动，享受购物乐趣。

2. 商家操作说明：

(1) 商品管理：

上传商品图片和信息，进行商品的上架和下架操作。
管理商品分类和库存，查看商品评价。

(2) 订单管理：

查看并处理用户订单，管理订单的发货和退货。
处理订单的支付和结算，确保订单正常完成。

(3) 发起直播：

通过腾讯云服务发起直播，展示商品并与用户互动。

管理直播间信息，确保直播顺利进行。

3. 管理员操作说明：

(1) 用户管理：

查看和管理用户信息，处理用户反馈和投诉。

管理用户权限，确保系统安全。

(2) 商家管理：

审核和管理商家信息，处理商家申请和反馈。

管理商家权限，确保商家行为合规。

6.3 项目效果



图 6-1 商城首页



图 6-2 商品列表

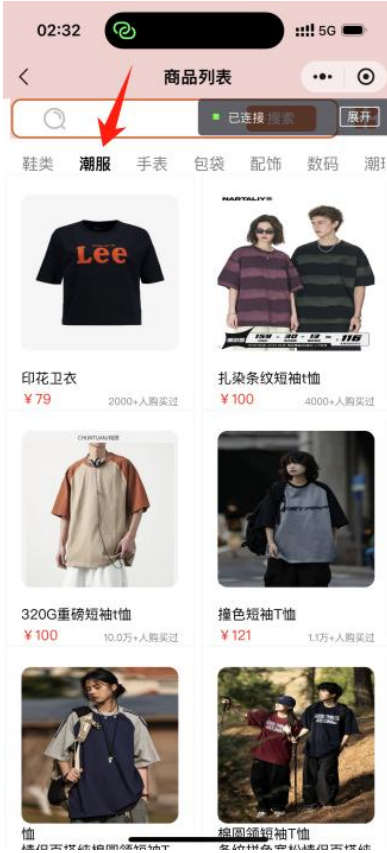


图 6-3 商品列表中的分类



图 6-4 商品详情页



图 6-5 商品细节页



图 6-6 展开商品的所有评论



图 6-7 选择规格

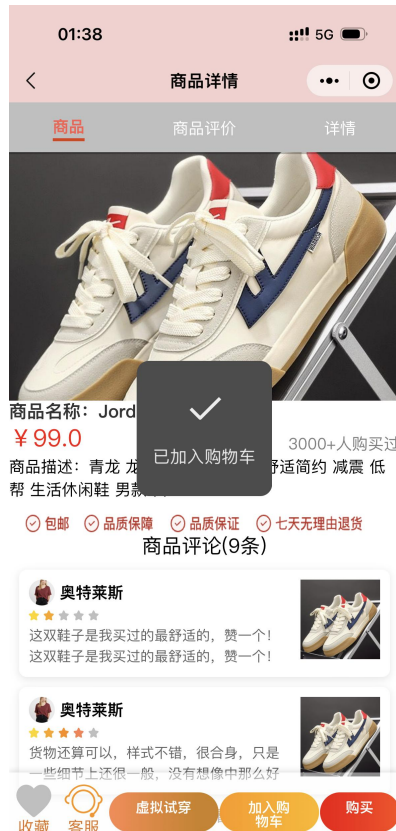


图 6-8 加入购物车成功



图 6-9 地址管理页

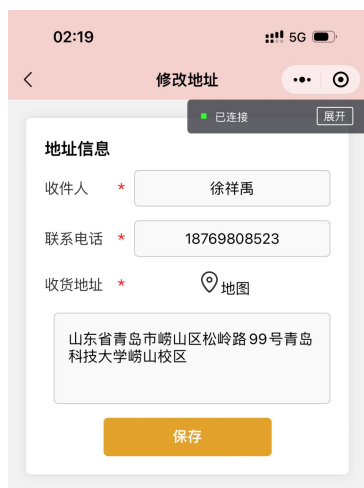


图 6-10 新增地址

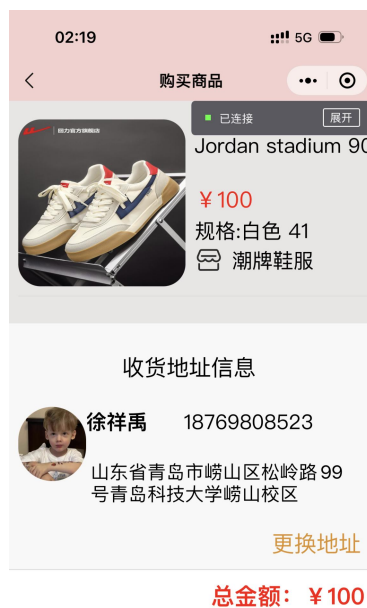


图 6-11 提交订单



图 6-12 商品分类



图 6-13 虚拟试穿鞋子



图 6-14 原图



图 6-15 虚拟试穿上的鞋子



图 6-16 展示鞋的提示点



图 6-17 客服聊天



图 6-18 聊天页面



图 6-19 查看新品



图 6-20 筛选条件

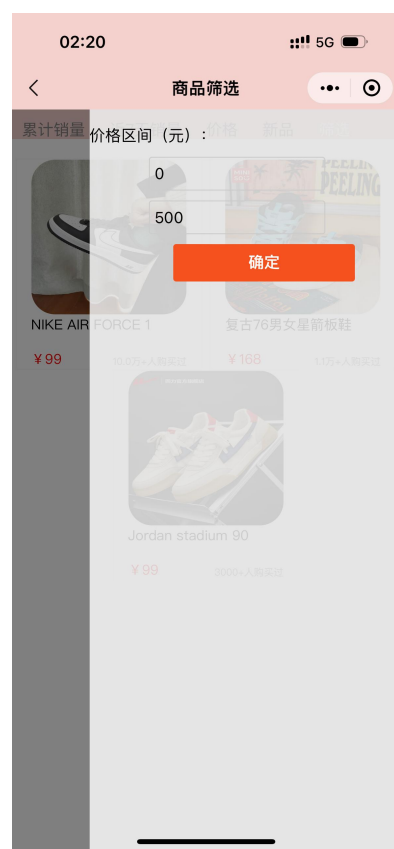


图 6-21 设置区间

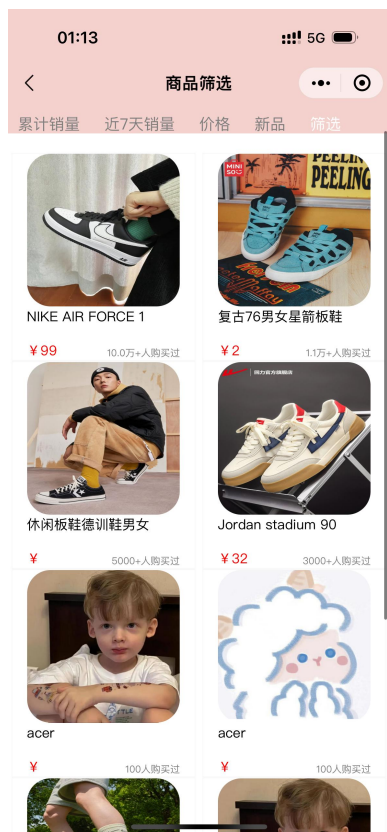


图 6-22 筛选后的商品



图 6-23 进入直播



图 6-24 观看直播

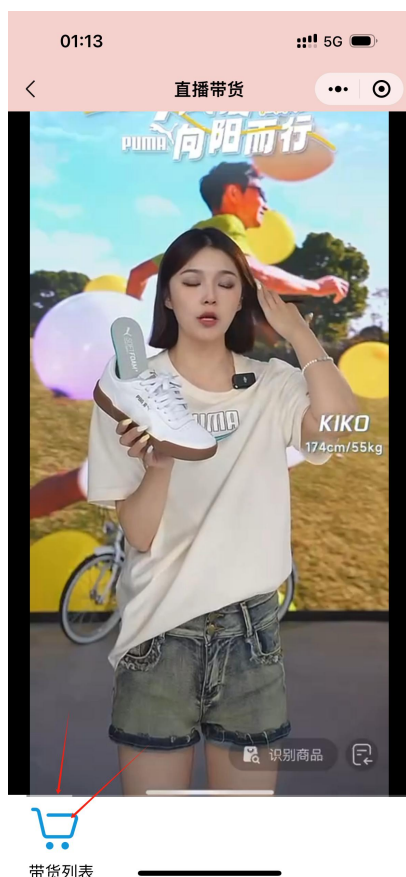


图 6-25 点击带货列表

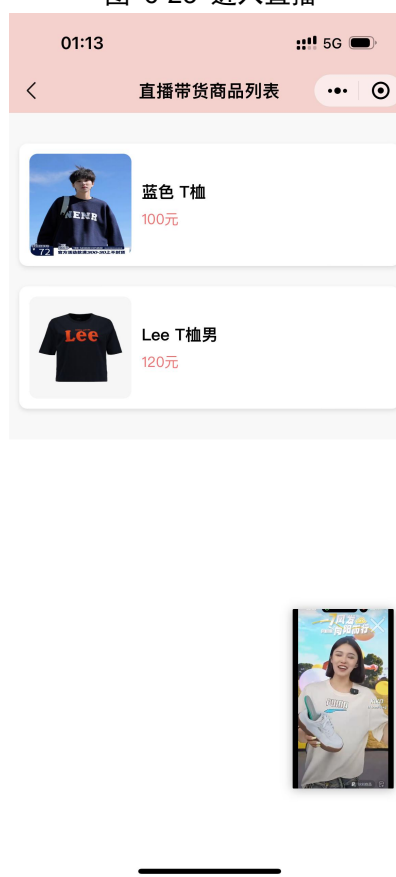


图 6-26 列表详情



图 6-27 购买带货商品



图 6-28 同城购物页面



图 6-29 显示商家距离

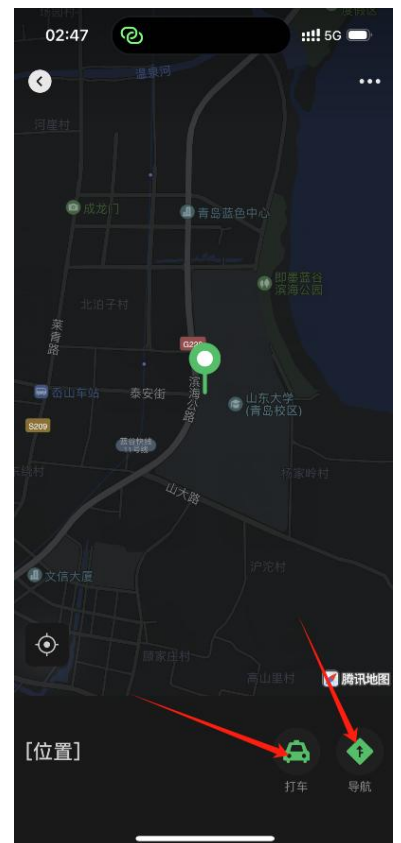


图 6-30 提供到店方式

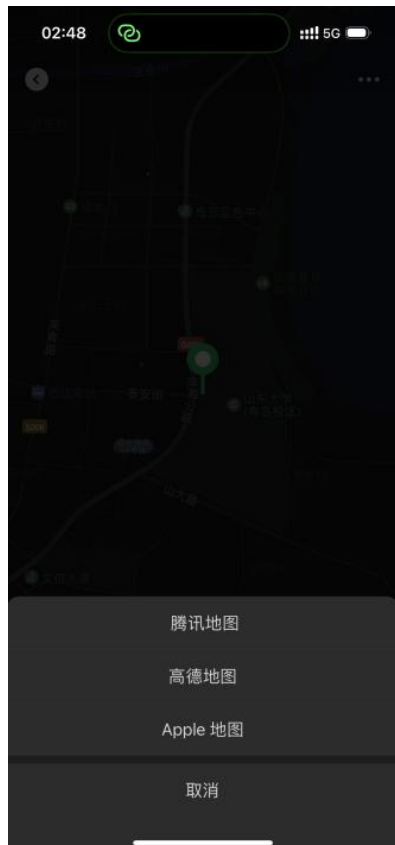


图 6-31 选项



图 6-32 到店方式

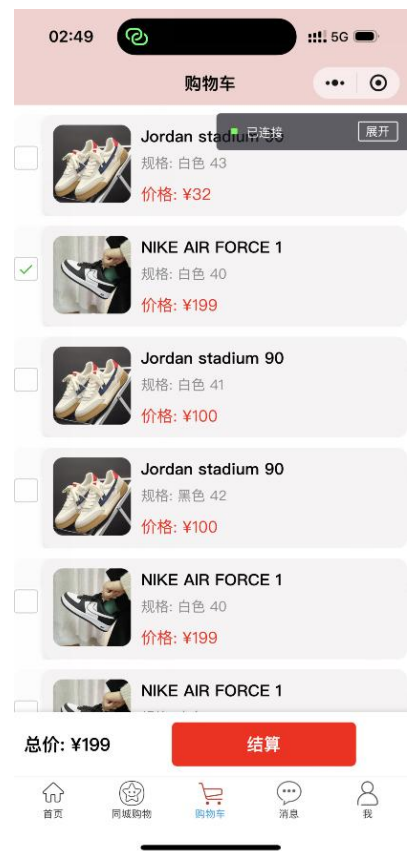


图 6-33 购物车界面



图 6-34 删除购物车中的商品



图 6-35 消息列表界面



图 6-36 个人中心界面

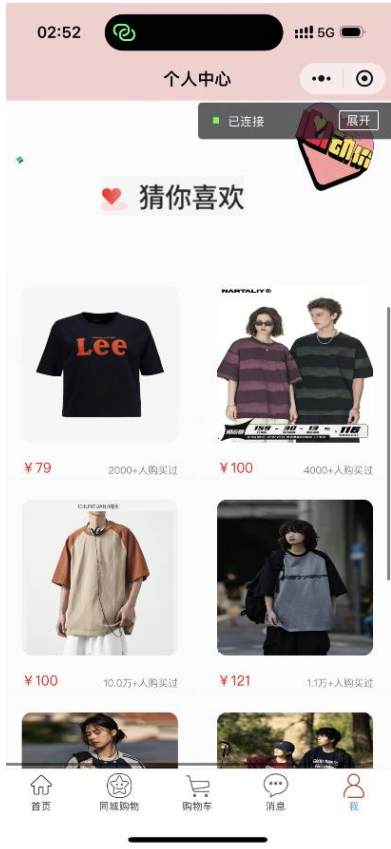


图 6-37 推荐界面



图 6-38 修改个人信息



图 6-39 全部订单



图 6-40 待收货的订单



图 6-41 待评价的订单



图 6-42 已完成的订单

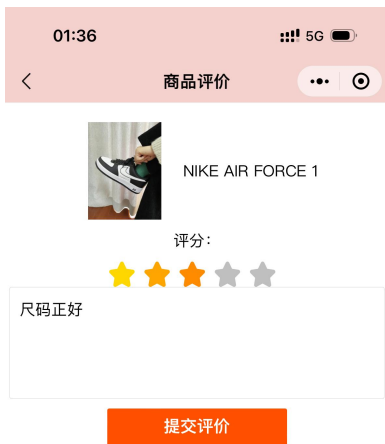


图 6-43 评价界面



图 6-44 虚拟试衣界面



图 6-45 虚拟试衣效果

第 7 章 系统测试

7.1 测试目的

本测试旨在确保小程序的质量和稳定性，测试小程序的功能、性能和用户体验，验证其是否满足开发需求，并发现并修复潜在问题。

7.2 测试环境

测试智慧商城小程序时，使用不同配置的移动设备打开微购小程序测试各项功能是否正常。测试手机的配置信息如表所示。

手机型号	操作系统	处理器	运行内存	微信版本
iPhone 15	iOS17	A16	6GB	8.0.48
iPhone 15 pro	iOS17	A17 Pro	8GB	8.0.45
小米 13	Android 13	高通骁龙 8 Gen 2	8GB	8.0.21
华为 P60	鸿蒙 3.1	高通骁龙 8+4G	8GB	8.0.48

7.3 功能测试

1. 商品分类查询与搜索功能测试

用户在智慧商城中使用分类查询或者搜索商品功能的测试，包括商品的分类检索以及商品的查询功能测试。描述见表所示。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-01001	无	1. 点击底部“分类”按钮； 2. 测试一级分类是否正常； 3. 点击一级分类名称，测试是否加载出	1. 进入分类展示界面； 2. 左侧边栏展示完整的一级分类列表； 3. 二级分类正确	通过

		二级分类; 4. 点击二级分类名称, 测试是否正确加载该分类商品;	加载; 4. 分类商品正确显示;	
w-user-01002	用户已进入分类界面	1. 点击顶部搜索框, 测试是否能正常输入搜索关键词; 2. 点击“搜索”按钮后, 测试是否显示搜索结果	1. 搜索框输入正常; 2. 正确显示关键词的输入结果;	通过

2. 购物车管理功能测试

测试用户在智慧商城小程序中使用购物车的功能是否正常, 具体为添加商品到购物车、从购物车结算及编辑购物车商品等功能, 详细描述见表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-02001	用户静默登录成功	1. 随意添加若干件商品到购物车, 测试是否能正常加入; 2. 点击底部“购物车”按钮进入购物车管理界面; 3. 滑动某商品, 选择删除, 测试是否能删除商品; 4. 修改某个商品的数量, 测试数量修改是否成功; 5. 选中购物车中的部分商品结算, 测试订单中商品是否正确;	1. 商品能正确加入到购物车; 2. 能够打开小程序的购物车管理界面; 3. 能够正常移除商品; 4. 能够修改某商品的数量; 5. 选中的部分商品与订单中的商品信息一致;	通过

3. 订单查询功能测试

测试智慧商城小程序中订单查询功能，具体为订单的分类查询及订单的详细信息查询，详细描述见表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-03 001	用户静默登录成功	1. 点击底部“我的”，进入个人中心； 2. 点击“全部订单”按钮； 3. 分别点击“待付款”、“待发货”“待收货”、“已完成”按钮，测试是否能正确显示订单信息； 4. 点击某一订单，测试是否显示订单详情；	1. 正确显示个人中心界面； 2. 进入订单列表显示界面； 3. 能够正确显示当前条件下的订单； 4. 能够正确显示该订单详情；	通过

4. 下单与结算功能测试

测试智慧商城小程序创建订单与结算订单功能，具体为从购物车创建订单、从商品购买页面创建订单、订单的支付结算、订单支付后状态改变和订单支付金额核算等功能。详细描述见表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-04 001	用户静默登录成功	1. 随意挑选一款商品，点击“立即购买”进入订单创建界面； 2. 从购物车选择若干商品，点击“结算”按钮； 3. 点击创建订单界面顶部的地址栏，测试新增收货地址或者选择已有收货地址功能是否正常；	1. 能够正确显示订单创建界面； 2. 能够正确显示订单创建界面； 3. 选择已有收货地址功能正常，能够新增收货地址； 4. 创建订单界面	通过

		4. 创建订单界面显示当前商品可用优惠券, 测试优惠券使用是否合法 5. 点击底部“提交订单”按钮, 测试是否创建成功并进入到待付款界面	显示的可用优惠券合法; 5. 获取到订单号并在待付款中显示订单;	
w-user-04002	已创建订单但未支付	1. 点击“我的”按钮, 点击“待付款”按钮, 进入到待付款订单界面; 2. 点击某个订单下方的“支付”按钮, 测试是否支付成功; 3. 完成支付后, 进入“待发货”订单界面, 测试订单是否支付成功:	1. 成功进入待付款订单界面 2. 显示支付成功 3. 支付后订单状态变更成功	通过

5. AR 试鞋功能测试

测试智慧商城小程序 AR 试鞋功能, 用户点击鞋子商品详情页面的“虚拟试穿”按钮, 上传或拍摄脚部照片, 系统展示试鞋效果。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-05001	用户授权访问相机和存储。	1. 打开小程序, 进入支持虚拟试鞋的商品详情页。 2. 点击“虚拟试鞋”按钮。 3. 允许小程序访问相机和存储权限 (如果未授权)。 4. 将手机对准脚部拍摄或上传已有脚部照片。	1. 用户能顺利进入虚拟试鞋功能页面。 2. 相机和存储权限申请正常, 用户能授权访问。 3. 系统能准确识别脚部, 并展示虚拟试穿效果	通过

		5. 系统加载虚拟试鞋功能，展示虚拟试鞋效果。 6. 更换不同的鞋子款式，观察虚拟试穿效果。 7. 选择一款满意的鞋子，添加到购物车。	果。 4. 不同鞋子款式的虚拟试穿效果展示正确。 5. 用户能顺利将选择的鞋子添加到购物车。	
--	--	---	--	--

6. AI 换衣功能测试

测试智慧商城小程序 AI 换衣功能，用户点击衣服商品详情页面的“虚拟试穿”按钮，上传或拍摄全身照片，系统展示试衣效果。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-06001	用户授权访问相机和存储。	1. 打开小程序，进入支持 AI 换衣的商品详情页。 2. 点击“AI 换衣”按钮。 3. 允许小程序访问相机和存储权限（如果未授权）。 4. 拍摄全身照片或上传已有全身照片。 5. 系统加载 AI 换衣功能，展示虚拟试穿效果。 6. 更换不同的衣服款式，观察虚拟试穿效果。 7. 选择一款满意的衣服，添加到购物车。	1. 用户能顺利进入 AI 换衣功能页面。 2. 相机和存储权限申请正常，用户能授权访问。 3. 系统能准确识别用户全身，并展示虚拟试穿效果。 4. 不同衣服款式的虚拟试穿效果展示正确。 5. 用户能顺利将选择的衣服添加到购物车。	通过

7. 客服聊天功能测试

测试智慧商城小程序客服聊天功能，用户打开客服聊天界面，发送消息，客服接收并回复。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-07 001	无	1. 用户选择商品并选择“联系客服”按钮。 2. 在聊天窗口输入消息并发送。 3. 客服接收消息并进行回复。 4. 用户查看回复，并继续与客服进行对话。 5. 结束对话，关闭聊天窗口。	1. 用户能顺利进入客服聊天窗口。 2. 消息发送和接收正常，无延迟或丢失。 3. 客服能及时回复用户消息，提供帮助。 4. 聊天结束后窗口正常关闭，消息记录保存。	通过

8. 直播模块功能测试

测试智慧商城小程序直播模块功能，商家发起直播，用户进入直播页面观看直播视频并与商家互动。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-08 001	家已开启直播，用户网络连接正常，带宽满足视频直播要求。	1. 用户进入小程序并选择“观看直播”按钮。 2. 选择正在进行的直播，点击进入。 3. 观看直播内容。 4. 在直播互动区输入评论或提问。	1. 用户能顺利进入直播观看页面。 2. 直播视频流畅播放，清晰度良好，无卡顿。 3. 评论和提问	通过

		5. 商家在直播中回复用户评论或提问。 6. 购买直播中推荐的商品，添加至购物车。 7. 结束直播，退出观看页面。	互动正常，商家及时回复。 4. 直播推荐商品能正常添加至购物车。 5. 直播结束后，用户能正常退出观看页面。	
--	--	---	--	--

9. 同城购物功能测试

测试智慧商城小程序同城购物功能，用户浏览同城购物页面商品，选择物流配送或到店自取方式，下单并支付。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-09001	系统已获取用户位置，并根据位置推荐同城商品。 同城商家已开设店铺，商品信息已上传。	1. 用户进入小程序并选择“同城购物”按钮。 2. 系统根据用户位置推荐同城商品列表。 3. 用户选择商品并查看详情。 4. 选择配送方式（物流配送或到店自取）。 5. 若选择到店自取，系统推荐合适的网约车或外卖服务。 6. 用户确认购买，选择支付方式并完成支付。 7. 查看订单详情，跟踪配送状态或到店自取信息。	1. 用户能顺利进入同城购物页面。 2. 系统推荐的同城商品准确，无误。 3. 商品详情页面显示正常，信息完整。 4. 用户能顺利选择配送方式并完成购买。 5. 到店自取功能正常，网约车和外卖服务推荐准确。	通过

			6. 订单详情和配送状态显示正确。	
--	--	--	-------------------	--

7.4 性能测试

在智慧商城小程序中，后台接口可分为公开接口和非公开接口。首页中展示的 banner、主题、商品和分类等都从公开 API 请求数据，为了提高响应速度，公开接口不需要签名。表 7-1 为利用 ApacheJMeter 模拟用户并发访问部分公开接口的测试结果。

表 7-1 模拟用户并发访问部分接口的测试结果

并发数	总请求数	平均值	最小值	最大值	99%响应时间	吞吐量	异常占比
500	62997	436	4	1349	815	858/s	0%
1000	390093	773	3	2135	1424	897/s	0%
1500	220123	1095	5	3317	2258	860/s	0%
2000	56835	1414	3	3161	2517	850/s	0%
2200	70315	1530	4	3205	2734	855/s	0%
2500	61075	2187	11	4331	3403	824/s	0%
3000	149969	2472	3	5248	4528	838/s	0%

从表 8-2 所示测试结果得出结论，接口平均响应时间与并发数表现为正相关关系。并发数为 1000 时，系统吞吐量为 897，99%用户响应时间小于 2 秒。当并发数增长到 1500 时，接口平均响应时间小于 2 秒，然而 99%用户响应时间大于 2 秒。测试结果表明，1000 位用户并发访问时，后台服务器能在 2 秒内正确响应，符合需求分析的执行效率指标。

7.5 总结

通过以上性能测试和功能测试，我们确认智慧商城小程序具备良好的性能表现和功能完整性，能够为用户提供稳定流畅的购物体验。系统在高并发情况下运行稳定，响应迅速，所有功能模块均能正常使用，满足用户需求。

第 8 章 总结

8.1 克服的困难

1. 微信小程序对非商户号的限制：由于我们没有商户号，无法使用支付接口和直播等功能。

问题：无法通过微信支付接口进行支付和直播功能的实现。

解决方案：对于直播功能，我们使用了腾讯云服务，利用其推流地址和视频接收地址来实现直播功能。

2. 用户体验：为了确保用户有良好的体验，我们反复优化前端页面和交互设计，进行了多轮测试和反馈修改，最终达到了理想的用户体验效果。

问题：初期用户界面设计缺乏用户友好性，交互体验不佳。

解决方案：我们进行了多轮用户测试，收集了大量用户反馈，并根据反馈不断改进界面设计和交互逻辑。同时，采用了前端性能优化技术，如图片懒加载、代码分片等，提升了页面加载速度。

3. 复杂功能实现：实现 AR 试鞋和 AI 换衣功能、实时客服聊天、同城购物等复杂功能，需要掌握先进的技术和算法。通过查阅大量文献资料 and 不断实验，我们最终成功实现了这些功能。

问题：AR 和 AI 技术的实现和集成。

解决方案：我们选用了改进的第三方技术，如 VisionKit 和 VOfitFusion 模型，来实现 AR 试鞋和 AI 换衣功能。深入学习了这些技术的原理和应用方法，通过多次实验和调试，最终实现了这些功能的稳定运行。

4. 性能优化：面对大量用户访问和高并发请求，系统性能优化成为一个关键问题。我们通过优化数据库查询、使用 Redis 缓存、进行代码优化等手段，提高了系统的响应速度和稳定性。

问题：高并发情况下，数据库查询和写入操作的性能瓶颈。

解决方案：我们采用了数据库读写分离策略，将读操作分散到多个从库，减轻主库压力。此外，使用 Redis 缓存热点数据，减少数据库访问频率。对于复杂查询，我们优化了 SQL 语句，并创建了必要的索引。

5. **技术整合**: 由于项目涉及多个技术栈, 包括微信小程序框架、Spring Boot、Flask、MyBatis、Redis 等, 我们通过详细的技术方案设计和不断的调试, 最终实现了技术的无缝整合。

问题: 不同技术栈之间的接口设计和数据传递。

解决方案: 我们设计了统一的 API 接口标准, 以确保各模块之间的数据传输顺畅。

8.2 升级演进

为了保持智慧商城小程序的竞争力和用户满意度, 我们规划了以下升级演进方向:

功能扩展: 增加更多实用功能, 如积分系统、社交分享等, 提升用户的互动性和粘性。

智能推荐: 优化推荐算法, 结合用户的浏览历史、购买行为和社交关系, 提供更加精准的商品推荐。

多平台支持: 扩展支持其他小程序平台和移动应用, 覆盖更多用户群体。

数据分析: 引入大数据分析技术, 深入分析用户行为和市场趋势, 提供数据驱动的运营决策支持。

安全升级: 持续加强系统的安全防护措施, 保障用户数据和交易安全。

8.3 商业推广

在商业推广方面, 我们将采取以下策略:

线上推广: 通过社交媒体、搜索引擎优化 (SEO)、内容营销等手段, 提升智慧商城小程序的品牌知名度和用户流量。

线下推广: 与实体店合作, 通过联合促销活动和广告宣传, 吸引更多线下用户。

用户激励: 实施用户激励计划, 如邀请好友奖励、消费积分兑换、优惠券发放等, 激发用户的使用和分享热情。

合作伙伴: 与物流公司、支付平台、知名品牌等建立战略合作, 提供更优质的服务和资源。

持续改进：根据用户反馈和市场变化，持续改进和优化产品，保持竞争优势。

8.4 感悟

在智慧商城项目中，我承担了多个重要角色和任务，从需求分析到前端开发、算法设计以及功能与性能测试的全流程负责。这一过程不仅增强了我对系统架构和技术实现的理解，还深化了我在用户体验和功能优化方面的能力。特别是通过整合 AR 试鞋和 AI 换衣功能，我学习并实践了如何将前沿技术应用于实际商业场景，这为我未来的技术探索和创新奠定了坚实基础。同时，功能与性能测试的经验也让我意识到系统稳定性和效率的重要性，为今后在软件开发领域的进一步探索奠定了坚实基础。

参考文献

- [1] 谭台哲,陈宏才,杨卓.VTON-FG: 通过图像边缘轮廓特征引导的虚拟试衣网络[J/OL]. 计算机工程与应用,1-11[2024-07-07].<http://kns.cnki.net/kcms/detail/11.2127.TP.20240624.1748.013.html>.
- [2] 张文悦,贾子彦,李青,等.基于 ResNet 和 UNet 的胃癌病理图像诊断系统[J/OL]. 河北大学学报(自然科学版):1-8[2024-07-07].<http://kns.cnki.net/kcms/detail/13.1077.n.20240628.1604.026.html>.
- [3] <https://blog.csdn.net/universsky2015/article/details/136835215>